Dr. Soumen Banerjee Senior Member-IEEE
Faculty – Eastern International University, Vietnam



Data Preprocessing

Introduction to Machine Learning with Python

Data Preprocessing in Machine Learning

- Pandas Tutorial
- Python NumPy
- SciPy Tutorial
- Scikit Learn Tutorial
- Matplotlib Tutorial
- Python Seaborn Tutorial
- TensorFlow

1 Python NumPy

Python NumPy

Numpy is a general-purpose array-processing package. It provides a high-performance multidimensional array object and tools for working with these arrays. It is the fundamental package for scientific computing with Python. Besides its obvious scientific uses, Numpy can also be used as an efficient multi-dimensional container of generic data.

Arrays in Numpy

A NumPy array is a structured collection of elements of the same data type stored in a table format.

- The number of dimensions is called the rank and the size along each dimension is called the shape.
- In NumPy, arrays are called ndarray and elements are accessed using square brackets [], often created from nested Python lists.

Creating a Numpy Array

Arrays in Numpy can be created by multiple ways, with various number of Ranks, defining the size of the Array. Arrays can also be created with the use of various data types such as lists, tuples, etc. The type of the resultant array is deduced from the type of the elements in the sequences.

```
import numpy as np

arr = np.array([1, 2, 3])
print(arr)

arr = np.array([[1, 2, 3],
                [4, 5, 6]])
print(arr)

arr = np.array((1, 3, 2))
print(arr)
```

Output

```
[1 2 3]
[[1 2 3]
 [4 5 6]]
[1 3 2]
```

Accessing the Array Index

In a numpy array, indexing or accessing the array index can be done in multiple ways. To print a range of an array, slicing is done. Slicing of an array is defining a range in a new array which is used to print a range of elements from the original array. Since, sliced array holds a range of elements of the original array, modifying content with the help of sliced array modifies the original array content.

```
import numpy as np

arr = np.array([[-1, 2, 0, 4],
                [4, -0.5, 6, 0],
                [2.6, 0, 7, 8],
                [3, -7, 4, 2.0]])

arr2 = arr[:2, ::2]
print ("first 2 rows and alternate columns(0 and 2):\n", arr2)

arr3 = arr[[1, 1, 0, 3],
                [3, 2, 1, 0]]
print ("\nElements at indices (1, 3), "
    "(1, 2), (0, 1), (3, 0):\n", arr3)
```

Output

```
first 2 rows and alternate columns(0 and 2):
 [[-1.  0.]
 [ 4.  6.]]

Elements at indices (1, 3), (1, 2), (0, 1), (3, 0):
 [0. 6. 2. 3.]
```

Basic Array Operations

In numpy, arrays allow a wide range of operations which can be performed on a particular array or a combination of Arrays. These operations include some basic Mathematical operation as well as Unary and Binary operations.

```
import numpy as np

a = np.array([[1, 2],
              [3, 4]])

b = np.array([[4, 3],
              [2, 1]])

print ("Adding 1 to every element:", a + 1)
print ("\nSubtracting 2 from each element:", b - 2)
print ("\nSum of all array elements: ", a.sum())
print ("\nArray sum:\n", a + b)
```

Output

```
Adding 1 to every element: [[2 3]
 [4 5]]

Subtracting 2 from each element: [[ 2  1]
 [ 0 -1]]

Sum of all array elements:  10

Array sum:
 [[5 5]
 [5 5]]
```

Data Types in Numpy

A NumPy array is a table of elements (usually numbers) of the same data type, indexed by a tuple of positive integers. Each array has a dtype that defines the type of its elements and how they are stored in memory. NumPy provides many numeric data types and you can either let NumPy guess the type or specify it explicitly when creating an array.

Constructing a Datatype Object

In Numpy, datatypes of Arrays need not to be defined unless a specific datatype is required. Numpy tries to guess the datatype for Arrays which are not predefined in the constructor function.

```
import numpy as np

x = np.array([1, 2])
print(x.dtype)

x = np.array([1.0, 2.0])
print(x.dtype)

x = np.array([1, 2], dtype = np.int64)
print(x.dtype)
```

Output

```
int64
float64
int64
```

Math Operations on DataType array

In Numpy arrays, basic mathematical operations are performed element-wise on the array.

These operations are applied both as operator overloads and as functions.

Many useful functions are provided in Numpy for performing computations on Arrays such as sum for addition of Array elements, T for Transpose of elements, etc.

```
import numpy as np

arr1 = np.array([[4, 7], [2, 6]],
                dtype = np.float64)

arr2 = np.array([[3, 6], [2, 8]],
                dtype = np.float64)

Sum = np.add(arr1, arr2)
print(Sum)

Sum1 = np.sum(arr1)
print(Sum1)

Sqrt = np.sqrt(arr1)
print(Sqrt)

Trans_arr = arr1.T
print(Trans_arr)
```

Output

```
[[ 7. 13.]
 [ 4. 14.]]
19.0
[[2.         2.64575131]
 [1.41421356 2.44948974]]
[[4. 2.]
 [7. 6.]]
```

2 Python Pandas

Pandas

Pandas is a high impact Python library designed for data manipulation and analysis. It streamlines the process of cleaning, transforming and analyzing large datasets with speed and precision. Seamlessly integrated with the Python ecosystem especially libraries like NumPy and Matplotlib it enables end to end data workflows.

Pandas operates around two core data structures:

- **Series (1D):** Used to handle single columns of data efficiently.
- **DataFrame (2D):** Used for structured, tabular data similar to spreadsheets or SQL tables.

Installing and Importing Pandas

To install Pandas, run the following command in terminal or command prompt | *pip install pandas*

Once installed, import Pandas into your Python script or notebook using the standard alias pd

```
import pandas as pd
```

Creating DataFrames

DataFrame in Pandas is a two dimensional table with labelled rows and columns, similar to an Excel sheet. Each column can store different types of data like numbers or text, which makes it flexible for real world data. You can easily create a DataFrame from lists, dictionaries or a list of dictionaries in Python.

Example: Creating a DataFrame using a Dictionary

In this example, a dictionary named data is created with keys representing column names (Name, Age) and values as lists containing the respective data. pd.DataFrame() function is then used to convert this dictionary into a DataFrame, which is stored in the variable df.

```
import pandas as pd

data = {
    'Name': ["Player_A", "Player_B", "Player_C"],
    'Age': [25, 30, 35]}

df = pd.DataFrame(data)

print(df)
```

Output:

```
...        Name  Age
0  Player_A   25
1  Player_B   30
2  Player_C   35
```

DataFrame

Creating Series

Series in Pandas is 1 dimensional labelled array capable of holding any data type (integers, strings, floats, etc.). Each element is associated with an index, either default (0, 1, 2...) or custom labels. You can easily create a series from lists, NumPy arrays, dictionaries or scalar values.

Example: Creating a Series using a List

```
import pandas as pd
age_series = pd.Series([25, 30, 35], index=["Player_A", "Player_B", "Player_C"])
print(age_series)
```

Output:

```
Player_A    25
Player_B    30
Player_C    35
dtype: int64
```

Series

Reading CSV and JSON Files

CSV (Comma Separated Values) files are simple text files used to store large datasets. In Python, Pandas provides the, read_csv() function, to load these files into a DataFrame. For our example we will use people.csv. By default, if the dataset is large, Pandas shows only the first 5 and last 5 rows. If you want to display the entire DataFrame, you can use to_string(). You can also control how many rows Pandas displays using the setting pd.options.display.max_rows.

```
import pandas as pd
df = pd.read_csv("people.csv")
print(df)
```

Output:

	First Name	Last Name	Sex	Email	Date of birth	Job Title
0	Shelby	Terrell	Male	elijah57@example.net	1945-10-26	Games developer
1	Phillip	Summers	Female	bethany14@example.com	1910-03-24	Phytotherapist
2	Kristine	Travis	Male	bthompson@example.com	1992-07-02	Homeopath
3	Yesenia	Martinez	Male	kaitlinkaiser@example.com	2017-08-03	Market researcher
4	Lori	Todd	Male	buchananmanuel@example.net	1938-12-01	Veterinary surgeon

Pandas Read CSV

Pandas provides flexible options to work with various file formats, including CSV and JSON (JavaScript Object Notation). Using read_json() function in pandas, we can read JSON files. To load a JSON file as a Pandas DataFrame, use the following code:

```
import pandas as pd

df = pd.read_json('data.json')
print(df)
```

Understanding and Analyzing the Data Frame

After creating or loading a DataFrame, inspecting and summarizing the data is an important step in understanding dataset. Pandas provides various functions to help you view and analyze the data.

- **head():** View the first n rows of the DataFrame (default is 5 rows).
- **tail():** View the last n rows of the DataFrame (default is 5 rows).
- **info():** This method provides a concise summary of the DataFrame, including the number of non null entries, column names and data types.
- **describe():** Generates summary statistics like count, mean and standard deviation for numerical columns. Use .describe(include='all') to include non numeric columns as well.

Let's see an example demonstrating the use of .head(), .tail(), .info() and .describe() methods:

```python
import pandas as pd

data = {'Name': ['Alice', 'Bob', 'Charlie', 'David', 'Eva'],
        'Age': [24, 27, 22, 32, 29],
        'City': ['New York', 'Los Angeles', 'Chicago', 'Houston', 'Phoenix']}
df = pd.DataFrame(data)

print("First 3 rows using head():")
print(df.head(3))

print("\nLast 2 rows using tail():")
print(df.tail(2))

print("\nDataFrame summary using info():")
df.info()
```

Output:

```
First 3 rows using head():
      Name  Age         City
0    Alice   24     New York
1      Bob   27  Los Angeles
2  Charlie   22      Chicago

Last 2 rows using tail():
    Name  Age     City
3  David   32  Houston
4    Eva   29  Phoenix

DataFrame summary using info():
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 5 entries, 0 to 4
Data columns (total 3 columns):
 #   Column  Non-Null Count  Dtype
---  ------  --------------  -----
 0   Name    5 non-null      object
 1   Age     5 non-null      int64
 2   City    5 non-null      object
dtypes: int64(1), object(2)
memory usage: 252.0+ bytes
```

Indexing in Pandas

Indexing in Pandas refers to process of accessing and selecting data from a Pandas DataFrame or Series. There are multiple ways to do this. We will cover how to basic indexing, select specific columns , apply slicing and use Boolean indexing to filter data efficiently.

Example : Basic Indexing(Selecting a single column) with use of [ ] operator:

```
import pandas as pd

data = {'Name': ['Alice', 'Bob', 'Charlie'],'Age': [25, 30, 35]}
df = pd.DataFrame(data)

age_column = df['Age']
print(age_column)
```

Output:

```
0    25
1    30
2    35
Name: Age, dtype: int64
```

Indexing

You can select multiple columns in a DataFrame by passing a list of column names. For example, using ['Name', 'Age'] will return both columns together as a new DataFrame .When selecting rows, Pandas provides two main methods:

- **.loc[]** : Used for label based indexing. You select rows using their index labels (the actual row names or index values).
- **.iloc[]** : Used for position based indexing. You select rows using their integer positions (like 0 for first row, 1 for second row).

```python
import pandas as pd

data = {'Name': ['Alice', 'Bob', 'Charlie'], 'Age': [25, 30, 35]}
df = pd.DataFrame(data)

row_by_label = df.loc[1]
row_by_position = df.iloc[1]

print("Row by label:\n", row_by_label)
print("\nRow by position:\n", row_by_position)
```

Output:

```
Row by label:
 Name    Bob
Age      30
Name: 1, dtype: object

Row by position:
 Name    Bob
Age      30
Name: 1, dtype: object
```

Indexing

Selecting and Filtering Data Based on Conditions

- Selecting and filtering data after indexing means narrowing down the dataset by applying conditions to choose only the rows or columns that meet specific criteria. Instead of using the entire dataset, we apply logical conditions to extract only the relevant data needed for analysis.
- In this example, we filter the rows where the Age column is greater than 30. This condition narrows the dataset to only those rows that satisfy the requirement.

```
import pandas as pd

data = {'Name': ['Alice', 'Bob', 'Charlie'],'Age': [25, 30, 35]}
df = pd.DataFrame(data)
filtered_df = df[df['Age'] > 28]

print(filtered_df)
```

Dealing with Rows and Columns in Pandas DataFrame

It involves various data manipulation techniques in Pandas, such as adding and deleting columns, truncating data, iterating over DataFrames and sorting data.

1. Adding a New Column to DataFrame: There are several methods available, each suitable for specific use cases. We can easily add new columns by assigning values to them by direct assignment.

```
import pandas as pd

df = pd.DataFrame({'Name': ['Alice', 'Bob', 'Charlie'], 'Age': [25, 30, 35]})

df['Salary'] = [50000, 60000, 70000]
print(df)
```

Output:

```
      Name  Age  Salary
0    Alice   25   50000
1      Bob   30   60000
2  Charlie   35   70000
```

Adding a new column

2. Renaming Columns in a DataFrame: There are multiple ways to rename columns depending on the requirement. We can use the rename() method for selective renaming of specific columns or directly modify the columns attribute when renaming all columns at once.

```
import pandas as pd

df = pd.DataFrame({'A': [1, 2, 3], 'B': [4, 5, 6], 'C': [7, 8, 9]})

df.rename(columns={'A': 'X', 'B': 'Y', 'C': 'Z'}, inplace=True)
print(df)
```

Output:

```
...    X  Y  Z
0      1  4  7
1      2  5  8
2      3  6  9
```

Renamed Columns

3. Reindexing Data with Pandas: Reindexing allows us to modify the row or column labels of a DataFrame or Series. This is useful for aligning data with a new set of labels, managing missing values and restructuring datasets. The reindex() method helps update row indices or column labels as needed.

Note: If the new index includes labels not present in the original DataFrame, the corresponding values will be set to NaN by default.

```python
import pandas as pd

data = {'A': [10, 20, 30], 'B': [40, 50, 60]}
df = pd.DataFrame(data)

new_index = [0, 1, 2, 3]
df_reindexed = df.reindex(new_index)
print("Reindexed Rows:\n", df_reindexed)
```

Output:

```
Reindexed Rows:
     A     B
0  10.0  40.0
1  20.0  50.0
2  30.0  60.0
3   NaN   NaN
```

Reindexed Rows

Handling Missing Data in Pandas

Working with Missing Data in Pandas is one of the most common tasks in data manipulation. Pandas provides various functions to identify, fill and drop missing values efficiently. To explore the procedure hand to hand in detail refer to below steps:

1. Identifying Missing Data With Pandas using isnull() and notnull():

- **isnull():** Returns True where values are missing (NaN) and False where values are present.
- **notnull():** Returns True where values are present and False where they are missing.

```python
import pandas as pd
import numpy as np
df = pd.DataFrame({'Col1': [1, 2, np.nan],'Col2': [3, np.nan, np.nan]})

print(df.isnull())
```

Output:

```
...     Col1   Col2
0  False  False
1  False   True
2   True   True
```

Checking Missing Values

When working with real world datasets, missing values are common. Broadly, we have two main ways to deal with missing data:

- Impute (Fill) the missing values
- Remove (Delete) the missing values

2. Filling Missing Data: To replace missing values with a specific value, we use the fillna() method.

For example: If you want to fill all missing values with a default value like 0, you can use df.fillna(0). Also we use inplace=True to make changes in our dataset

```python
import pandas as pd
import numpy as np

df = pd.DataFrame({'Col1': [1, 2, np.nan],'Col2': [3, np.nan, np.nan]})
df.fillna(0,inplace=True)

print(df)
```

Output:

```
...     Col1  Col2
0   1.0   3.0
1   2.0   0.0
2   0.0   0.0
```

Filled Missing Values with 0

3. Dropping Missing Data With Pandas: We can also remove rows or columns that contain missing values using .dropna().

We can remove missing data in different ways depending on the situation. Pandas provides flexible options that allow us to either remove entire columns or selectively drop rows based on specific conditions. Some common approaches include:

- Removing columns that contain missing values using df.dropna(axis=1)
- Dropping rows where certain columns have missing values using df.dropna(subset=['Column1', 'Column2'])

```
import pandas as pd

data = {'Name': ['Alice', 'Bob', None], 'Age': [25, None, 35]}
df = pd.DataFrame(data)
df_dropped = df.dropna()

print(df_dropped)
```

Output:

```
    Name   Age
0  Alice  25.0
```

Missing Data Dropped

Aggregation and Grouping With Pandas

Aggregation and grouping in Pandas are powerful tools for analyzing and summarizing data. Grouping allows to segment your data into categories, while aggregation performs operations (like sum, mean, or count) on these groups to derive insights. groupby() function is commonly used for grouping data, followed by aggregation methods like sum(), mean() or custom functions for statistical analysis.

Example:

- **groupby('Category'):** Groups data by the 'Category' column.
- **.sum(), .mean(), and .size():** Perform aggregation on the grouped data.

```
import pandas as pd

data = {'Category': ['A', 'B', 'A', 'B'], 'Value': [10, 20, 30, 40]}
df = pd.DataFrame(data)

grouped_sum = df.groupby('Category').sum()
print("Sum:\n", grouped_sum)

grouped_mean = df.groupby('Category')['Value'].mean()
print("\nMean:\n", grouped_mean)

grouped_agg = df.groupby('Category').agg(['sum', 'mean'])
print("\nAggregated:\n", grouped_agg)
```

```
Sum:
            Value
Category
A              10
B              20
C              30
D              40

Mean:
 Category
A    10.0
B    20.0
C    30.0
D    40.0
Name: Value, dtype: float64

Aggregated:
          Value
           sum  mean
Category
A           10  10.0
B           20  20.0
C           30  30.0
D           40  40.0
```

Merging and Concatenating Data in Pandas

This section covers techniques for combining multiple DataFrames or Series into a single DataFrame. These operations are essential for integrating datasets and can be performed in several ways:

1. Merging DataFrames: Combines data based on common column or index using functions like merge or join. There are mainly 4 types of joins:

- **Inner Join:** Keeps rows that match in both DataFrames.
- **Left Join:** Keeps all rows from the left DataFrame and match data from the right.
- **Right Join and Outer Join:** Keep similar but differ in data retention rules.

```python
import pandas as pd

data1 = {
    'Name': ['Alice', 'Charlie', 'Edward', 'Grace'],
    'Years_Experience': [2, 3, 4, 6]
}
df1 = pd.DataFrame(data1)

data2 = {

    'Name': ['Alice', 'Charlie', 'Frank', 'Grace'],
    'Role': ['Manager', 'Analyst', 'Developer', 'HR']
}
df3 = pd.DataFrame(data2)

df_merged_inner = pd.merge(df1, df3, on='Name', how='inner')

df_merged_outer = pd.merge(df1, df3, on='Name', how='outer')

print("\nInner Merge (Intersection of Names):\n", df_merged_inner)
print("\nOuter Merge (Union of Names):\n", df_merged_outer)
```

Output:

```
Inner Merge (Intersection of Names):
      Name  Years_Experience     Role
0    Alice                 2  Manager
1  Charlie                 3  Analyst
2    Grace                 6       HR

Outer Merge (Union of Names):
      Name  Years_Experience       Role
0    Alice               2.0    Manager
1  Charlie               3.0    Analyst
2   Edward               4.0        NaN
3    Frank               NaN  Developer
4    Grace               6.0         HR
```

Merging in Pandas

2. Concatenating Data: Refers to stacking DataFrames either vertically (adding rows) or horizontally (adding columns). This can be achieved using the pd.concat() function. Let's create two dataFrames and concatenate it with the original one:

Example:

- By default, pd.concat() concatenates along rows (axis=0), which means the new DataFrame is stacked vertically below the original one.
- ignore_index=True ensures that the index is reset, creating a continuous new index instead of keeping the original indices from both DataFrames.

```python
import pandas as pd

df = pd.DataFrame({
    'Name': ['Alice', 'Charlie', 'Edward', 'Grace'],
    'Years_Experience': [2, 3, 4, 6],
    'Role': ['Manager', 'Analyst', 'Developer', 'HR']
})


new_df = pd.DataFrame({
    'Name': ['John', 'Lily'],
    'Years_Experience': [5, 3],
    'Role': ['Designer', 'Developer']
})


concatenated_df = pd.concat([df, new_df], ignore_index=True)

print("Concatenated DataFrame:\n")
print(concatenated_df)
```

Output:

```
Concatenated DataFrame:

      Name  Years_Experience       Role
0    Alice                 2    Manager
1  Charlie                 3    Analyst
2   Edward                 4  Developer
3    Grace                 6         HR
4     John                 5   Designer
5     Lily                 3  Developer
```

Concatenated Dataframe

Reshaping Data in Pandas

Reshaping data involves changing the structure of rows and columns to better organize or analyze data. Common operations include:

1. Pivot Tables in Pandas: Reshape data based on column values. The .pivot_table() method is particularly powerful for creating aggregated views of data.

```
import pandas as pd

data = {'Date': ['2024-10-01', '2024-10-01', '2024-10-02', '2024-10-02'],'Category': ['A', 'B', 'A', 'B'],'Values':
[10, 20, 15, 25]}
df = pd.DataFrame(data)

pivot_table = df.pivot_table(values='Values', index='Date', columns='Category', aggfunc='sum')
print(pivot_table)
```

Output:

Category	A	B
Date		
2024-10-01	10	20
2024-10-02	15	25

Pivoting in Pandas

2. Melting : Multiple columns are combined into a single key value pair.

When you melt data:

- The column names become the keys.
- The values in the columns become the values.

Note: To melt data, you must specify columns that act as "identifiers" (id_vars) and others that need to be melted (value_vars).

```
import pandas as pd

data = {
    'Date': ['2024-10-01', '2024-10-01', '2024-10-02', '2024-10-02'],
    'Category_A': [10, None, 15, None],
    'Category_B': [None, 20, None, 25]
}

df = pd.DataFrame(data)

melt_df = df.melt(id_vars='Date',
                  var_name='Category',
                  value_name='Value')

print(melt_df)
```

Output:

```
...          Date    Category  Value
0  2024-10-01  Category_A   10.0
1  2024-10-01  Category_A    NaN
2  2024-10-02  Category_A   15.0
3  2024-10-02  Category_A    NaN
4  2024-10-01  Category_B    NaN
5  2024-10-01  Category_B   20.0
6  2024-10-02  Category_B    NaN
7  2024-10-02  Category_B   25.0
```

Melting in Pandas

3. Stacking and Unstacking With Pandas: Reshape the data by changing its rows and columns, particularly when working with MultiIndex DataFrames. These techniques help pivot the layout of your data.

- **Stacking**: Moves the data from columns to rows (making the DataFrame taller).
- **Unstacking**: Moves the data from rows to columns (making the DataFrame wider). Let's take a simple DataFrame with multiple levels of columns.

Example:

When columns are stacked into rows using stack(), the DataFrame structure changes. The data now has a MultiIndex in the rows:

- First level Product (e.g., Product_A, Product_B)
- Second level Category (e.g., Sales, Expenses)
- The quarter values (Q1, Q2) remain as columns, while the higherlevel column labels move into the row index.

```python
import pandas as pd
data = {
    ('Sales', 'Q1'): [100, 150],
    ('Sales', 'Q2'): [200, 250],
    ('Expenses', 'Q1'): [50, 70],
    ('Expenses', 'Q2'): [80, 90]
}

index = ['Product_A', 'Product_B']
df = pd.DataFrame(data, index=index)
print(df)

stacked_df = df.stack()
print(stacked_df)

unstacked_df = stacked_df.unstack()
print(unstacked_df)
```

Output:

```
...          Sales      Expenses
                Q1   Q2       Q1  Q2
Product_A      100  200       50  80
Product_B      150  250       70  90
               Sales  Expenses
Product_A Q1      100        50
          Q2      200        80
Product_B Q1      150        70
          Q2      250        90
          Sales      Expenses
             Q1   Q2       Q1  Q2
Product_A   100  200       50  80
Product_B   150  250       70  90
```

Stacking and Unstacking in Pandas

Handling Large Datasets With Pandas

When working with large datasets or computationally intensive tasks, performance optimization becomes essential. Efficient data handling ensures faster processing and better memory management. Below are some practical techniques to improve workflow efficiency in Pandas.

1. **Working with Chunks of Data**: When dealing with large datasets that may not fit into memory, you can use the `chunksize` parameter in `pd.read_csv()` to load the data in chunks that allows to process the data in manageable pieces rather than loading everything at once.
2. **Optimizing Memory Usage by Changing Data Types**: By default, Pandas may use larger data types than necessary (e.g., `int64` instead of `int8`), which can lead to inefficient memory usage. Changing the data types with `.astype()` can optimize memory consumption significantly, especially with large datasets.

3 Scikit-learn

Scikit-learn

Scikit-learn is an open-source Python library that simplifies the process of building machine learning models. It offers a clean and consistent interface that helps both beginners and experienced users work efficiently.

- Supports tasks like classification, regression, clustering and preprocessing
- Makes model building fast and reliable
- Provides ready-to-use tools for training and evaluation
- Reduces complexity by avoiding manual implementation of algorithms

Installing and Using Scikit-learn

Before we start building models we need to install **Scikit-learn.** It requires Python 3.8 or newer and depends on two important libraries: NumPy and SciPy. Make sure these are installed first.

To install Scikit-learn run the following command: | *pip install -U scikit-learn*

Learning Model Building in Scikit-learn

Step 1: Loading a Dataset

A dataset consists of:

- **Features (X):** Input variables that describe the data
- **Target (y):** The value we want to predict

Scikit-learn provides built-in datasets like Iris, Digits and Boston Housing. Using the Iris dataset:

- load_iris() loads the data
- X stores feature data
- y stores target labels
- feature_names and target_names give descriptive names

We can inspect the first few rows to understand the structure. For custom datasets, Pandas is commonly used to load external files such as CSVs.

```python
from sklearn.datasets import load_iris
iris = load_iris()

X = iris.data
y = iris.target

feature_names = iris.feature_names
target_names = iris.target_names

print("Feature names:", feature_names)
print("Target names:", target_names)

print("\nType of X is:", type(X))

print("\nFirst 5 rows of X:\n", X[:5])
```

Output:

```
Feature names: ['sepal length (cm)', 'sepal width (cm)', 'petal length (cm)', 'petal width (cm)']
Target names: ['setosa' 'versicolor' 'virginica']

Type of X is: <class 'numpy.ndarray'>

First 5 rows of X:
 [[5.1 3.5 1.4 0.2]
 [4.9 3.  1.4 0.2]
 [4.7 3.2 1.3 0.2]
 [4.6 3.1 1.5 0.2]
 [5.  3.6 1.4 0.2]]
```

Loading dataset

Sometimes we need to work on our own custom data then we load an external dataset. For this we can use the pandas library for easy loading and manipulating datasets.

Step 2: Splitting the Dataset

To evaluate a model fairly, we split data into:

- Training set: Used to train the model
- Testing set: Used to evaluate how well the model generalizes

Using train_test_split, we split the Iris dataset so that 60% is for training and 40% for testing (test_size=0.4). random_state=1 ensures reproducibility.

After splitting, we get:

- X_train, y_train -> Training data
- X_test, y_test -> Testing data

Checking the shapes ensures the data is split correctly.

```
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.4, random_state=1)
```

Now lets check the **Shapes** of the Splitted Data to ensures that both sets have correct proportions of data avoiding any potential errors in model evaluation or training.

```
print("X_train Shape:",  X_train.shape)
print("X_test Shape:", X_test.shape)
print("Y_train Shape:", y_train.shape)
print("Y_test Shape:", y_test.shape)
```

Output:

```
X_train Shape: (90, 4)
X_test Shape: (60, 4)
Y_train Shape: (90,)
Y_test Shape: (60,)
```

Shape of Splitted Data

Step 3: Handling Categorical Data

Machine learning algorithms work with numerical inputs, so categorical (text) data must be converted into numbers. If not encoded properly, models can misinterpret categories. Scikit-learn provides multiple encoding methods:

1. Label Encoding: It converts each category into a unique integer. For example in a column with categories like 'cat', 'dog' and 'bird', it would convert them to 0, 1 and 2 respectively. This method works well when the categories have a meaningful order such as “Low”, “Medium” and “High”.

- **LabelEncoder():** It is initialized to create an encoder object that will convert categorical values into numerical labels.
- **fit_transform():** This method first fits the encoder to the categorical data and then transforms the categories into corresponding numeric labels.

```
from sklearn.preprocessing import LabelEncoder
categorical_feature = ['cat', 'dog', 'dog', 'cat', 'bird']
encoder = LabelEncoder()
encoded_feature = encoder.fit_transform(categorical_feature)
print("Encoded feature:", encoded_feature)
```

Output:

Encoded feature: [1 2 2 1 0]

2. One-Hot Encoding: One-Hot Encoding creates separate binary columns for each category. This is useful when categories do not have any natural ordering. Example: cat, dog, bird -> 3 new columns (cat/dog/bird) with 1s and 0s.

- Input must be reshaped into a 2D array
- OneHotEncoder(sparse_output=False) generates binary columns

```
from sklearn.preprocessing import OneHotEncoder
import numpy as np

categorical_feature = ['cat', 'dog', 'dog', 'cat', 'bird']

categorical_feature = np.array(categorical_feature).reshape(-1, 1)

encoder = OneHotEncoder(sparse_output=False)

encoded_feature = encoder.fit_transform(categorical_feature)

print("OneHotEncoded feature:\n", encoded_feature)
```

Output:

```
OneHotEncoded feature:
 [[0. 1. 0.]
 [0. 0. 1.]
 [0. 0. 1.]
 [0. 1. 0.]
 [1. 0. 0.]]
```

Step 4: Training the Model

Now that our data is ready, it's time to train a machine learning model. Scikit-learn has many algorithms with a consistent interface for training, prediction and evaluation. Here we'll use Logistic Regression as an example.

- **log_reg = LogisticRegression(max_iter=200):** Creating a logistic regression classifier object.
- **log_reg.fit(X_train, y_train):** Using this the logistic regression model adjusts the model's parameters to best fit the data.

```
from sklearn.linear_model import LogisticRegression
log_reg = LogisticRegression(max_iter=200)
log_reg.fit(X_train, y_train)
```

```
LogisticRegression
LogisticRegression(max_iter=200)
```

Training Using Logistic Regression.

Step 5: Make Predictions

Once trained we use the model to make predictions on the test data X_test by calling the predict method. This returns predicted labels y_pred.

- **log_reg.predict:** It uses trained logistic regression model to predict labels for the test data X_test.

```
y_pred = log_reg.predict(X_test)
```

Step 6: Evaluating Model Accuracy

Check how well our model is performing by comparing y_test and y_pred. Here we are using the metrics module's method accuracy_score.

```
from sklearn import metrics
print("Logistic Regression model accuracy:", metrics.accuracy_score(y_test, y_pred))
```

> **Output:**
>
> *Logistic Regression model accuracy: 0.9666666666666667*

Now we want our model to make predictions on new sample data. Then the sample input can simply be passed in the same way as we pass any feature matrix. Here we used it as sample = [[3, 5, 4, 2], [2, 3, 5, 4]]

```
sample = [[3, 5, 4, 2], [2, 3, 5, 4]]
preds = log_reg.predict(sample)
pred_species = [iris.target_names[p] for p in preds]
print("Predictions:", pred_species)
```

Output:

Predictions: [np.str_('virginica'), np.str_('virginica')]

Features of Scikit-learn

Scikit-learn is used because it makes building machine learning models straightforward and efficient. Here are some important reasons:

1. **Ready-to-Use Tools**: It provides built-in functions for common tasks like data preprocessing, training models and making predictions. This saves time by avoiding the need to code algorithms from scratch.
2. **Easy Model Evaluation**: With tools like cross-validation and performance metrics it helps to measure how well our model works and identify areas for improvement.
3. **Wide Algorithm Support**: It offers many popular machine learning algorithms including classification, regression and clustering which gives us flexibility to choose the right model for our problem.
4. **Smooth Integration**: Built on top of important Python libraries like NumPy and SciPy so it fits into our existing data analysis workflow.
5. **Simple and Consistent Interface**: The same straightforward syntax works across different models helps in making it easier to learn and switch between algorithms.
6. **Model Tuning Made Easy**: Tools like grid search help us fine-tune our model's settings to improve accuracy without extra hassle.

Benefits of using Scikit-learn

- **User-Friendly**: Scikit-learn's consistent and simple interface makes it accessible for beginners and best for experts.
- **Time-Saving:** Pre-built tools and algorithms reduce development time which allows us to focus more on solving problems than coding details.
- **Better Model Performance**: Easy-to-use tuning and evaluation tools helps in improving model accuracy and reliability.
- **Flexible and Scalable**: Supports a wide range of algorithms and integrates smoothly with other Python libraries helps in making it suitable for projects of any size.
- **Strong Community Support**: A large, active community ensures regular updates, extensive documentation and plenty of resources to help when we get stuck.

4 Matplotlib in Python

Matplotlib in Python

Matplotlib is a used Python library used for creating static, animated and interactive data visualizations. It is built on the top of NumPy and it can easily handles large datasets for creating various types of plots such as line charts, bar charts, scatter plots, etc.

Visualizing Data with Pyplot using Matplotlib

Pyplot is a module in Matplotlib that provides a simple interface for creating plots. It allows users to generate charts like line graphs, bar charts and histograms with minimal code.

1. Line Chart

Line chart is one of the basic plots and can be created using plot() function. It is used to represent a relationship between two data X and Y on a different axis.

Syntax: *matplotlib.pyplot.plot(x, y)*

Parameter: x, y Coordinates for data points.

Example: This code plots a simple line chart with labeled axes and a title using Matplotlib.

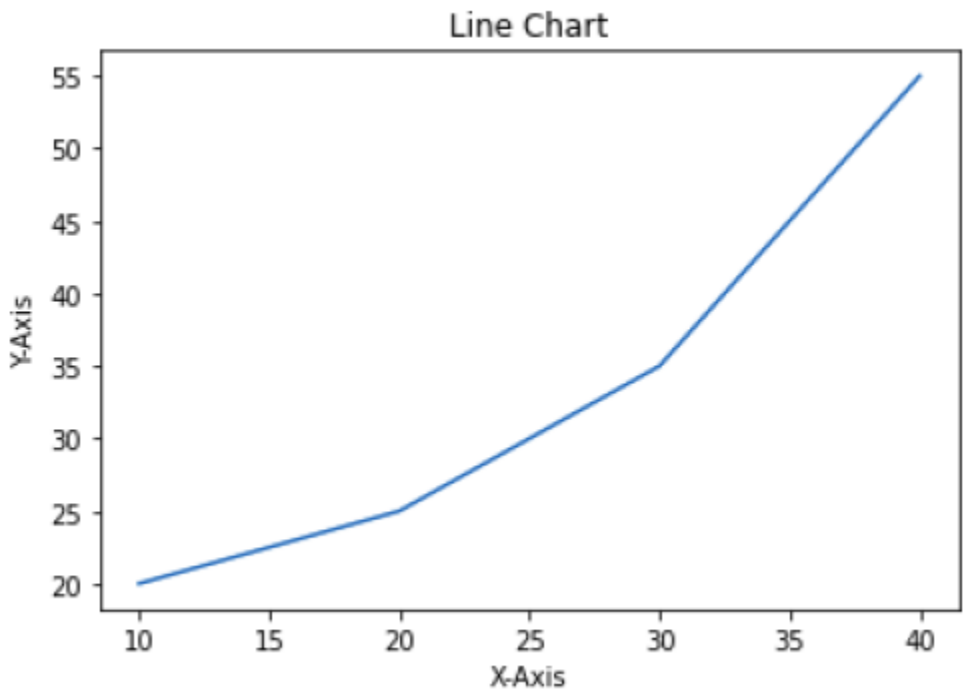
```
import matplotlib.pyplot as plt

x = [10, 20, 30, 40]

y = [20, 25, 35, 55]

plt.plot(x, y)
plt.title("Line Chart")
plt.ylabel('Y-Axis')
plt.xlabel('X-Axis')
plt.show()
```

Output



2. Bar Chart

Bar chart displays categorical data using rectangular bars whose lengths are proportional to the values they represent. It can be plotted vertically or horizontally to compare different categories.

Syntax: *matplotlib.pyplot.bar(x, height)*

Parameter:

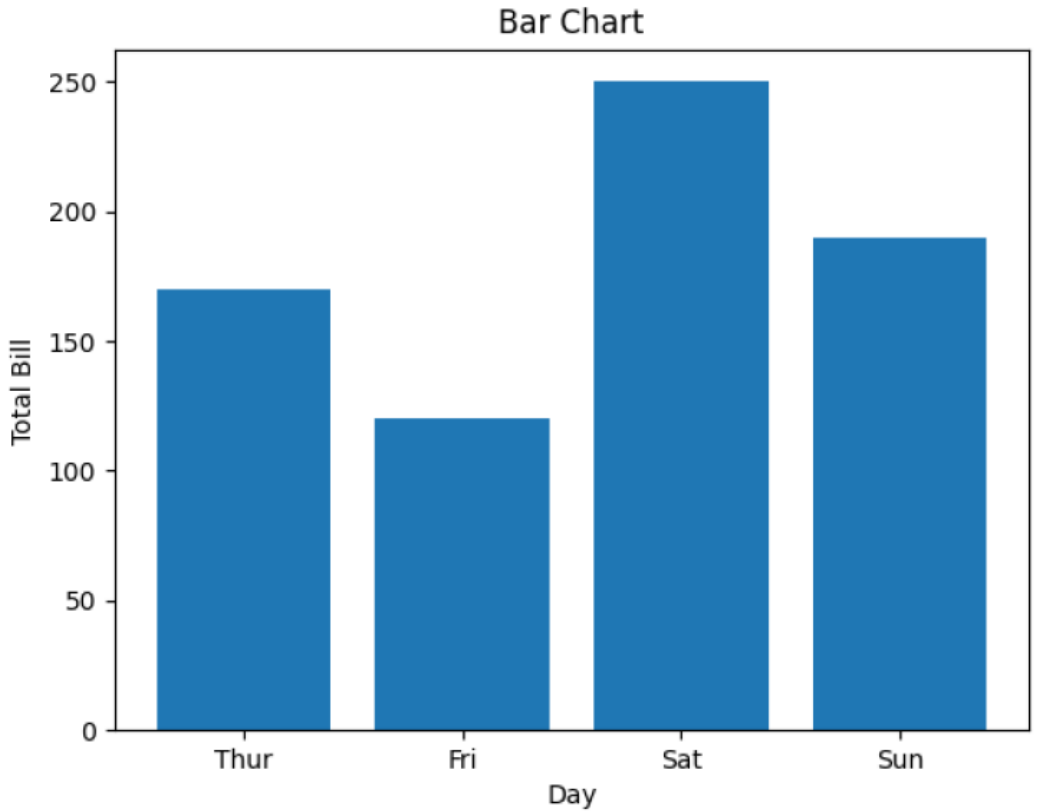
- **x:** Categories or positions on x-axis.
- **height:** Heights of the bars (y-axis values).

Example: This code creates a simple bar chart to show total bills for different days. X-axis represents the days and Y-axis shows total bill amount.

```
import matplotlib.pyplot as plt

x = ['Thur', 'Fri', 'Sat', 'Sun']
y = [170, 120, 250, 190]

plt.bar(x, y)
plt.title("Bar Chart")
plt.xlabel("Day")
plt.ylabel("Total Bill")
plt.show()
```



3. Histogram

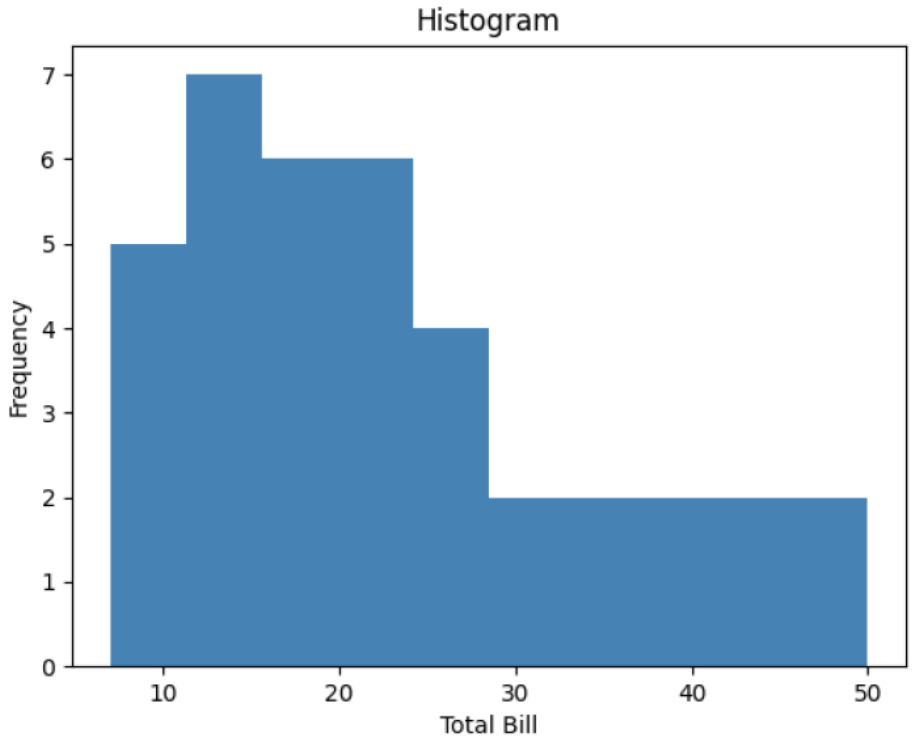
Histogram shows the distribution of data by grouping values into bins. The hist() function is used to create it, with X-axis showing bins and Y-axis showing frequencies.

Syntax: *matplotlib.pyplot.hist(x, bins=None)*

Parameter:

- **x:** Input data.
- **bins:** Number of bins (intervals) to group data.

Example: This code plots a histogram to show frequency distribution of total bill values from the list x. It uses 10 bins and adds axis labels and a title for clarity.




```
import matplotlib.pyplot as plt

x = [7, 8, 9, 10, 10, 12, 12, 12, 13, 14, 14, 15, 16, 16, 17, 18, 18, 19, 20, 20,
     21, 22, 23, 24, 25, 25, 26, 28, 30, 32, 35, 36, 38, 40, 42, 44, 48, 50]

plt.hist(x, bins=10, color='steelblue')
plt.title("Histogram")
plt.xlabel("Total Bill")
plt.ylabel("Frequency")
plt.show()
```

4. Scatter Plot

Scatter plots are used to observe relationships between variables. The scatter() method in the matplotlib library is used to draw a scatter plot.

Syntax: *matplotlib.pyplot.scatter(x, y)*

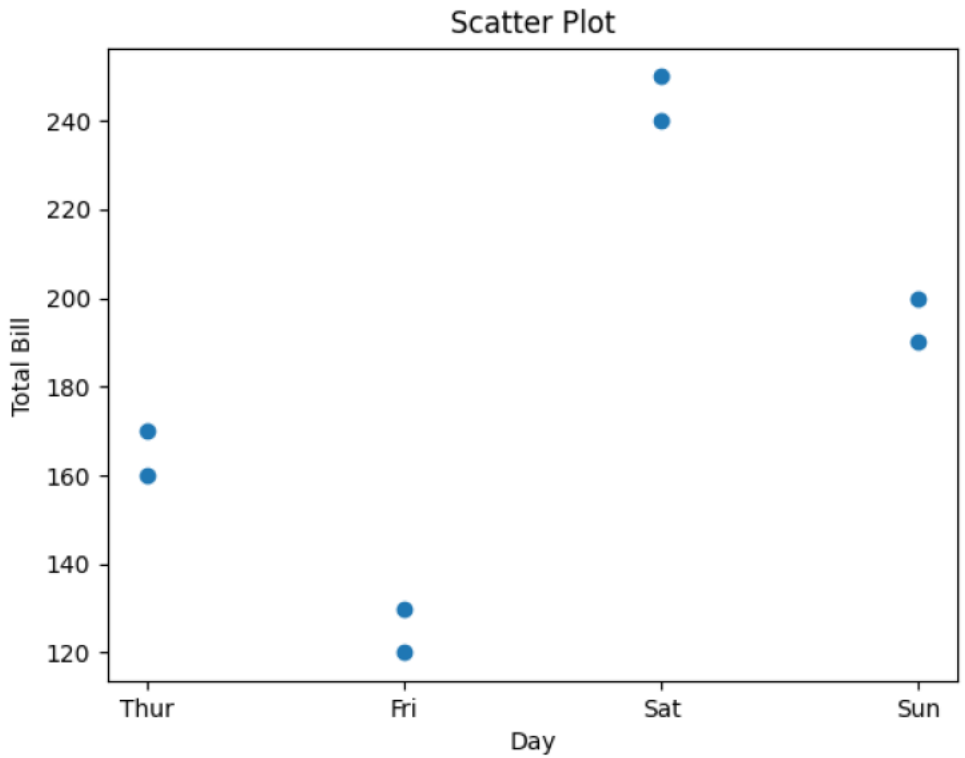
Parameter: x, y Coordinates of the points.

Example: This code creates a scatter plot to visualize the relationship between days and total bill amounts using scatter().

```
import matplotlib.pyplot as plt

x = ['Thur', 'Fri', 'Sat', 'Sun', 'Thur', 'Fri', 'Sat', 'Sun']
y = [170, 120, 250, 190, 160, 130, 240, 200]

plt.scatter(x, y)
plt.title("Scatter Plot")
plt.xlabel("Day")
plt.ylabel("Total Bill")
plt.show()
```



5. Pie Chart

Pie chart is a circular chart used to show data as proportions or percentages. It is created using the pie(), where each slice (wedge) represents a part of the whole.

Syntax: *matplotlib.pyplot.pie(x, labels=None, autopct=None)*

Parameter:

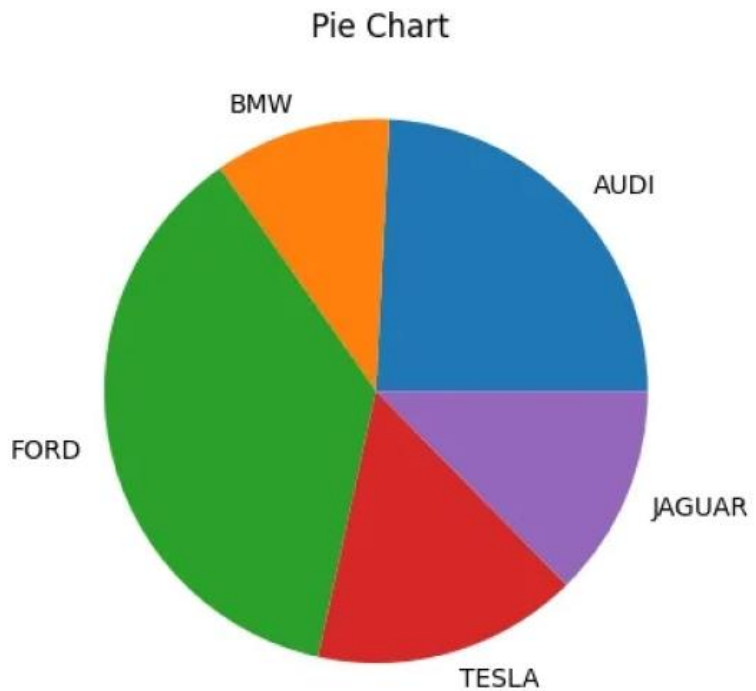
- **x:** Data values for pie slices.
- **labels:** Names for each slice.
- **autopct:** Format to display percentage (e.g., '%1.1f%%').

Example: This code creates a simple pie chart to visualize distribution of different car brands. Each slice of pie represents the proportion of cars for each brand in the dataset.

```
import matplotlib.pyplot as plt
import pandas as pd

cars = ['AUDI', 'BMW', 'FORD','TESLA', 'JAGUAR',]
data = [23, 10, 35, 15, 12]

plt.pie(data, labels=cars)
plt.title(" Pie Chart")
plt.show()
```



6. Box Plot

Box plot is a simple graph that shows how data is spread out. It displays the minimum, maximum, median and quartiles and also helps to spot outliers easily.

Syntax: *matplotlib.pyplot.boxplot(x, notch=False, vert=True)*

Parameter:

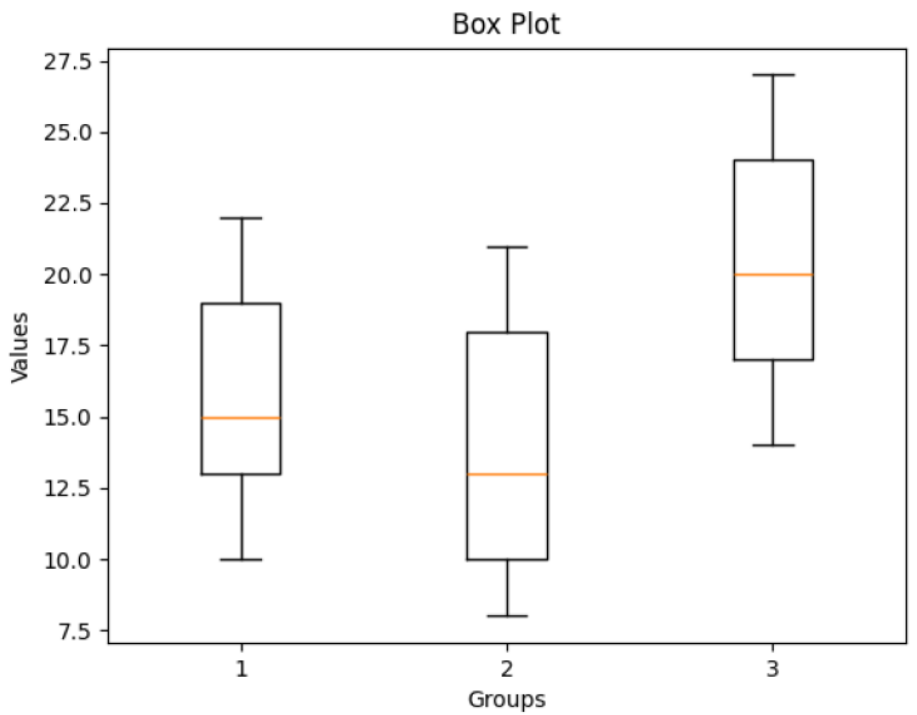
- **x:** Data for which box plot is to be drawn (usually a list or array).
- **notch:** If True, draws a notch to show the confidence interval around the median.
- **vert:** If True, boxes are vertical. If False, they are horizontal.

Example: This code creates a box plot to show the data distribution and compare three groups using matplotlib

```
import matplotlib.pyplot as plt

data = [ [10, 12, 14, 15, 18, 20, 22],
         [8, 9, 11, 13, 17, 19, 21],
         [14, 16, 18, 20, 23, 25, 27] ]

plt.boxplot(data)
plt.xlabel("Groups")
plt.ylabel("Values")
plt.title("Box Plot")
plt.show()
```



7. Heatmap

Heatmap is a graphical representation of data where values are shown as colors. It helps visualize patterns, correlations or intensity in a matrix-like format. It is created using **imshow()** method in Matplotlib.

Syntax: *matplotlib.pyplot.imshow(X, cmap='viridis')*

Parameter:

- **X:** 2D array (data to display as an image or heatmap).
- **cmap:** Sets the color map.

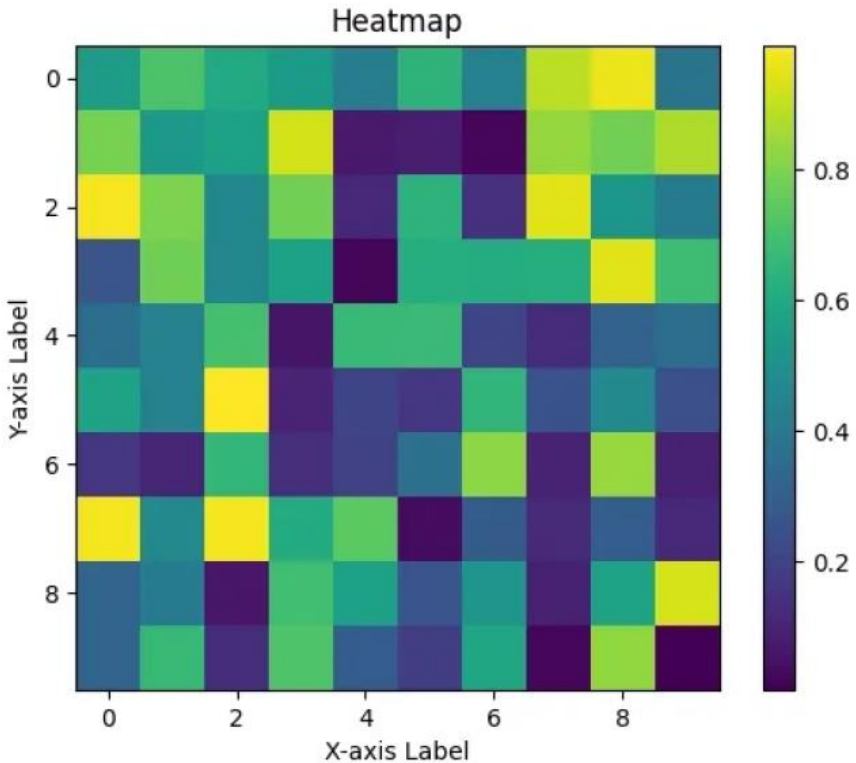
Example: This code creates a heatmap of random 10×10 data using imshow(). It uses 'viridis' color map and colorbar() adds a color scale.

```
import matplotlib.pyplot as plt
import numpy as np

np.random.seed(0)
data = np.random.rand(10, 10)

plt.imshow(data, cmap='viridis', interpolation='nearest')

plt.colorbar()
plt.xlabel('X-axis Label')
plt.ylabel('Y-axis Label')
plt.title('Heatmap')
plt.show()
```



Explanation:

- **np.random.seed(0):** Ensures same random values every time (reproducibility).
- **np.random.rand(10, 10):** Generates a 10×10 array of random numbers between 0 and 1.

How to Customize Matplotlib Visualizations

Customization in Matplotlib allows you to improve look and clarity of plots by adjusting elements like colors, styles, labels, titles and gridlines. It helps make visualizations more informative and visually appealing for better data communication.

1. Customizing Line Chart

Line charts can be customized using various properties:

1. **Color:** Change the color of the line
2. **Linewidth**: Adjust the width of the line
3. **Marker**: Change the style of plotted points
4. **Markersize**: Change the size of the markers
5. **Linestyle**: Define the style of the line like solid, dashed, etc.

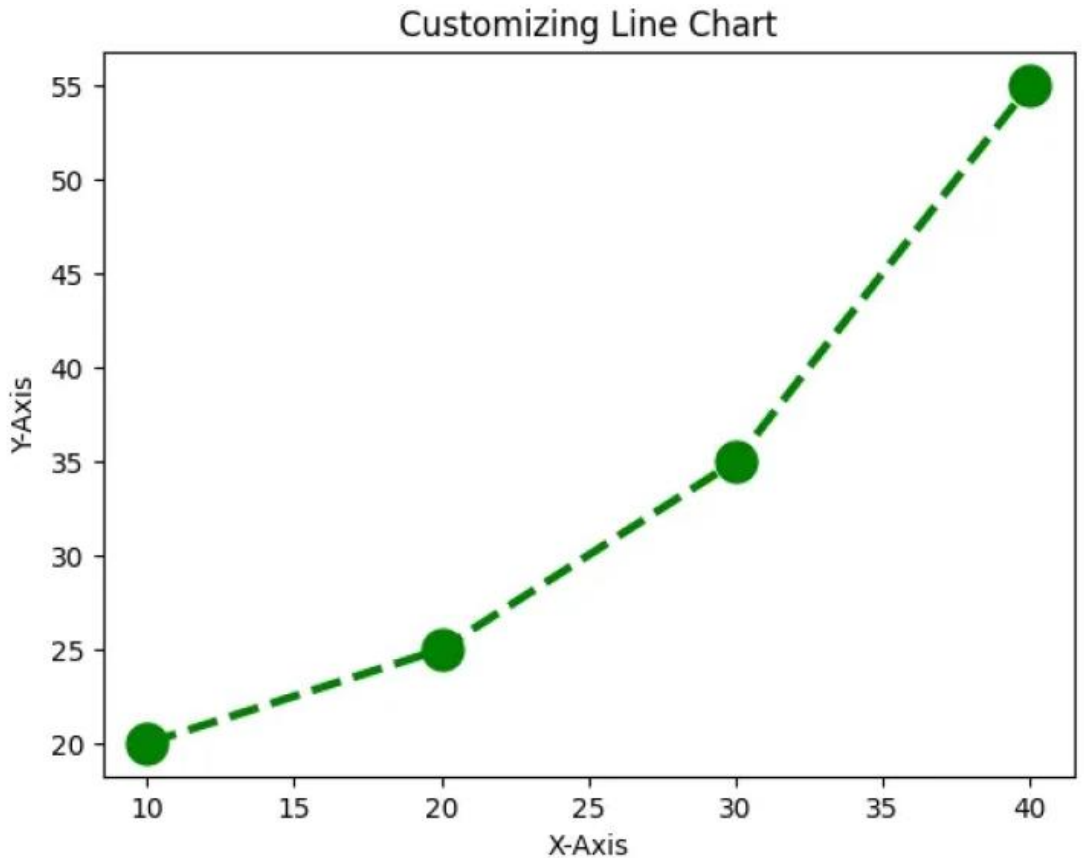
Example: This code creates a customized line chart with a green dashed line, thicker width, large circular markers and labeled axes and title.

```
import matplotlib.pyplot as plt

x = [10, 20, 30, 40]
y = [20, 25, 35, 55]

plt.plot(x, y, color='green', linewidth=3, marker='o', markersize=15, linestyle='--')

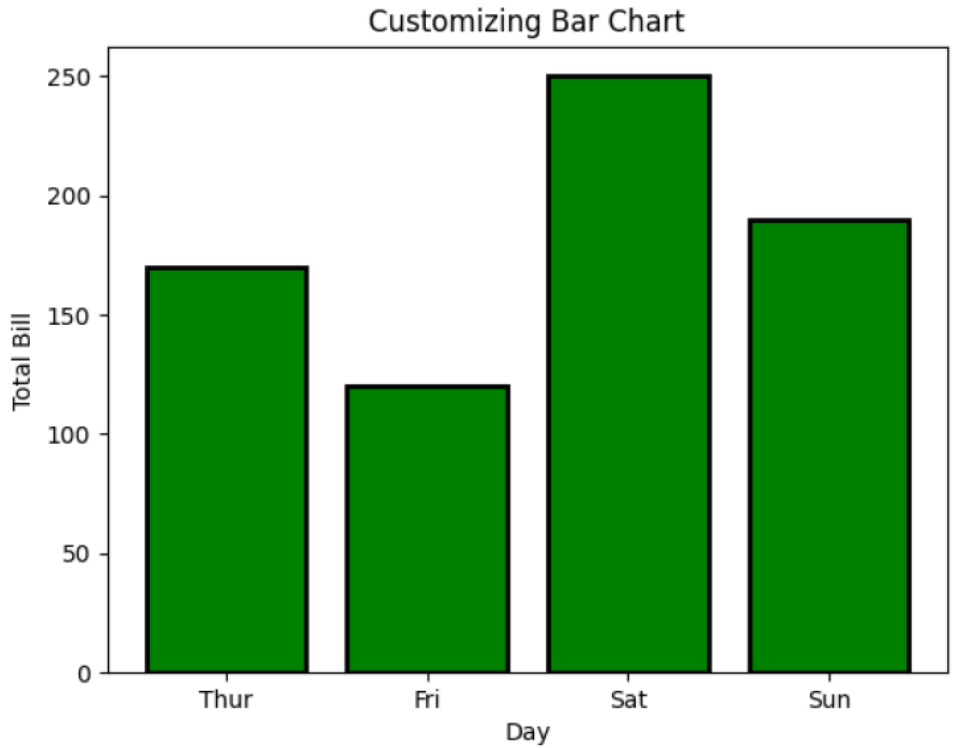
plt.title("Customizing Line Chart")
plt.ylabel('Y-Axis')
plt.xlabel('X-Axis')
plt.show()
```



2. Customizing Bar Chart

Bar charts can be made more informative and visually appealing by customizing:

- **Color:** Fill color of the bars
- **Edgecolor:** Color of the bar edges
- **Linewidth:** Thickness of the edges
- **Width:** Width of each bar



Example: This code creates a customized bar chart with green bars, blue edges, thicker border lines and labeled axes and title.

```
import matplotlib.pyplot as plt

x = ['Thur', 'Fri', 'Sat', 'Sun']
y = [170, 120, 250, 190]

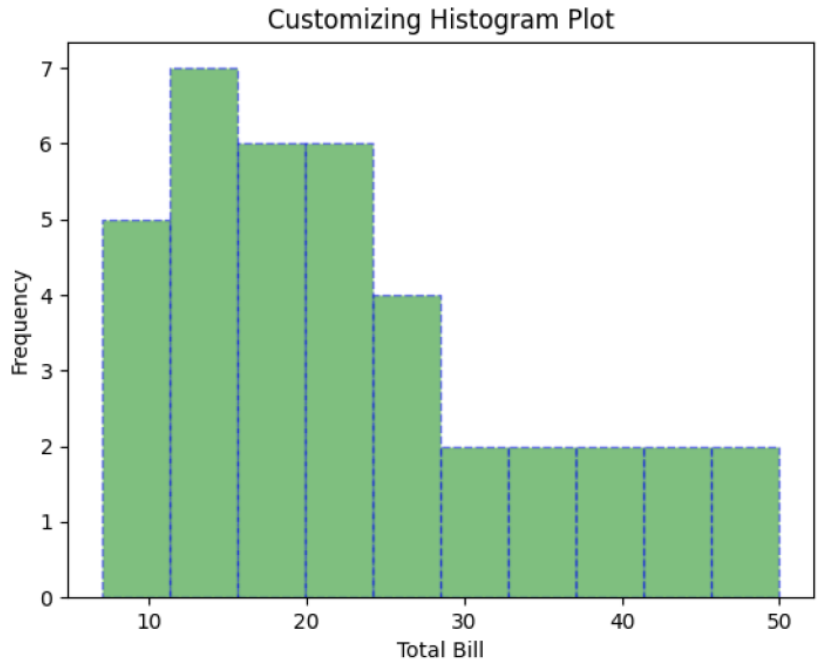
plt.bar(x, y, color='green', edgecolor='black', linewidth=2)

plt.title("Customizing Bar Chart")
plt.xlabel("Day")
plt.ylabel("Total Bill")
plt.show()
```

3. Customizing Histogram Plot

To make histogram plots more effective various customizations can be applied:

- **Bins:** Number of groups (bins) to divide data into
- **Color**: Bar fill color
- **Edgecolor**: Bar edge color
- **Linestyle**: Style of the edges like solid, dashed, etc.
- **Alpha**: Transparency level (0 = transparent, 1 = opaque)




```
import matplotlib.pyplot as plt

x = [7, 8, 9, 10, 10, 12, 12, 12, 13, 14, 14, 15, 16, 16, 17,
     18, 18, 19, 20, 20, 21, 22, 23, 24, 25, 25, 26, 28, 30,
     32, 35, 36, 38, 40, 42, 44, 48, 50]

plt.hist(x, bins=10, color='green', edgecolor='blue',linestyle='--', alpha=0.5)

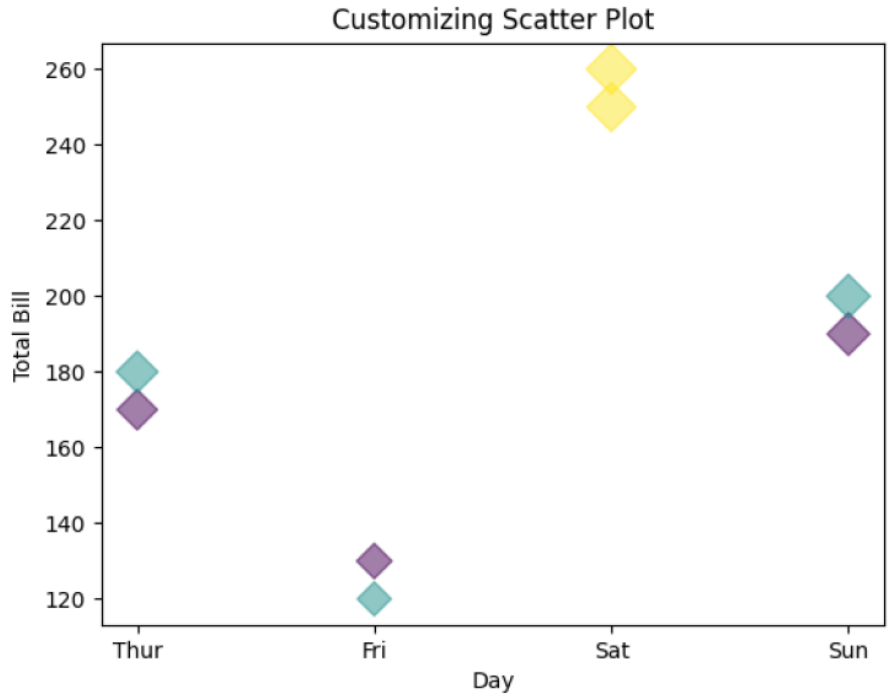
plt.title("Customizing Histogram Plot")
plt.xlabel("Total Bill")
plt.ylabel("Frequency")
plt.show()
```

Example: This code creates a customized histogram with green bars, blue edges, dashed border lines, semi-transparent fill and labeled axes and title.

4. Customizing Scatter Plot

Scatter plots can be enhanced with:

- **S:** Marker size (single value or array)
- **C:** Color of markers or sequence of colors
- **Marker:** Marker style like circle, diamond, etc.
- **Linewidths:** Width of marker borders
- **Edgecolor:** Color of marker borders
- **Alpha:** Blending value, between 0 (transparent) and 1 (opaque)




```
import matplotlib.pyplot as plt

x = ['Thur', 'Fri', 'Sat', 'Sun', 'Thur', 'Fri', 'Sat', 'Sun']
y = [170, 120, 250, 190, 180, 130, 260, 200]
size = [2, 3, 4, 2, 3, 2, 4, 3]
bill = [170, 120, 250, 190, 180, 130, 260, 200]

plt.scatter(x, y, c=size, s=bill, marker='D', alpha=0.5)

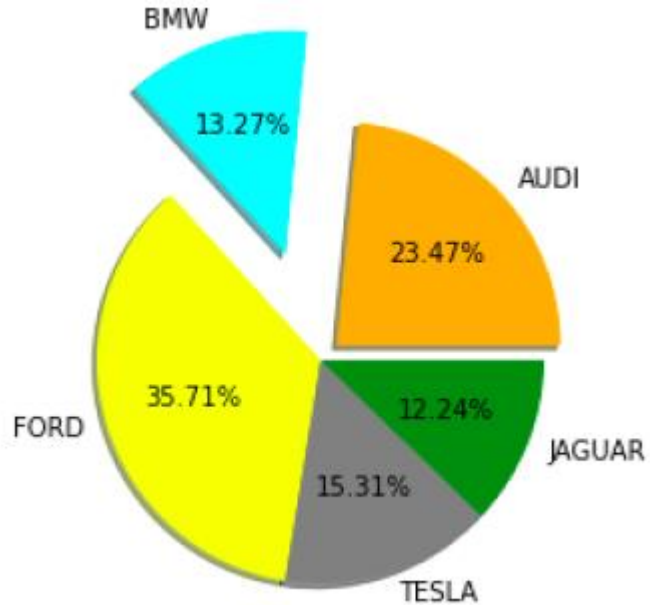
plt.title("Customizing Scatter Plot")
plt.xlabel("Day")
plt.ylabel("Total Bill")
plt.show()
```

Example: This code creates a customized scatter plot using diamond-shaped markers, where color represents size, marker size reflects the total bill and transparency is added for better visualization. It includes labeled axes and a title.

5. Customizing Pie Chart

To make pie charts more effective and visually appealing use:

- **Explode:** Moving the wedges of the plot
- **Autopct:** Label the wedge with their numerical value.
- **Color:** Colors of the slices
- **Shadow:** Used to create a shadow effect



Example: This code creates a customized pie chart with colored slices, exploded segments for emphasis, percentage labels with two decimal places and a shadow effect for better visual appeal.

```
import matplotlib.pyplot as plt
import pandas as pd

cars = ['AUDI', 'BMW', 'FORD','TESLA', 'JAGUAR',]
data = [23, 13, 35, 15, 12]
explode = [0.1, 0.5, 0, 0, 0]
colors = ( "orange", "cyan", "yellow","grey", "green",)

plt.pie(data, labels=cars, explode=explode, autopct='%1.2f%%',colors=colors, shadow=True)
plt.show()
```

Matplotlib's Core Components: Figures and Axes

1. Figure class

The Figure class represents the full drawing area or canvas that can hold one or more plots. It is created using the figure() function and lets user control the overall size, layout and background of the plot window.

Syntax: *matplotlib.figure.Figure(figsize=None, facecolor=None)*

Parameter:

- **figsize:** Sets size of the figure (width, height) in inches.
- **facecolor:** Sets background color of the figure.

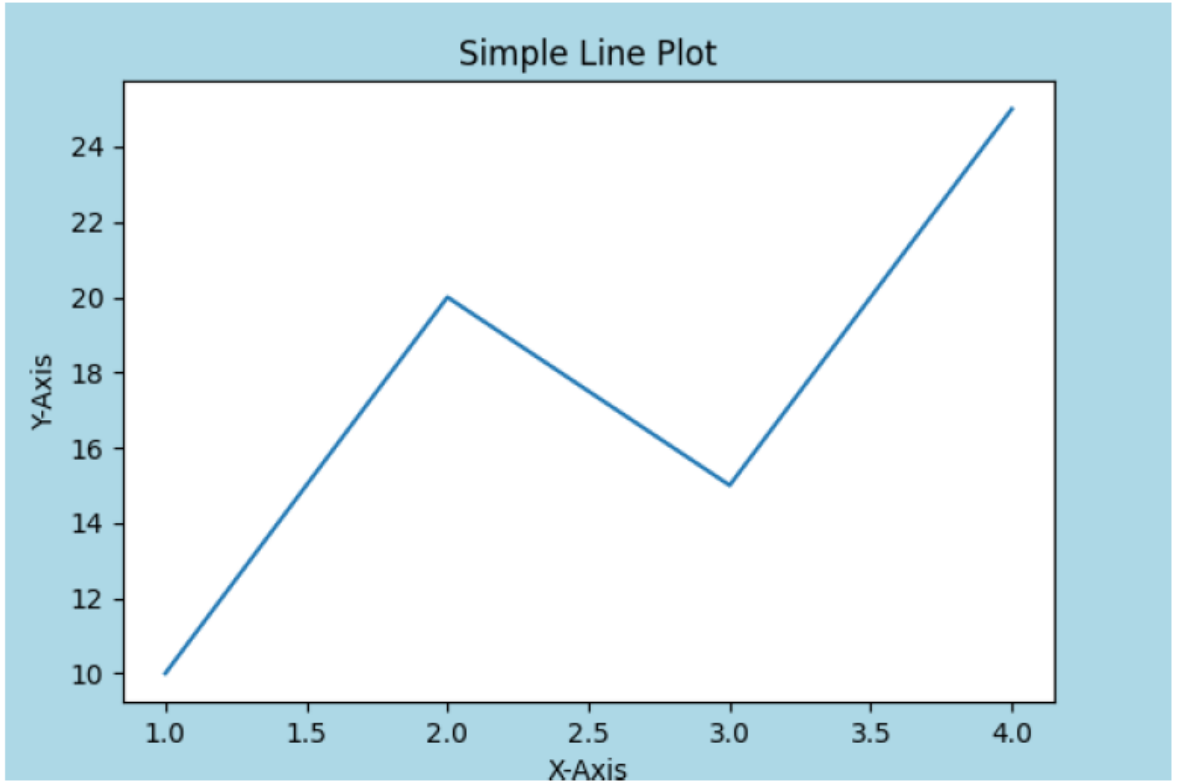
Example: This code demonstrates how to use Figure class to create a simple line plot. It sets figure size and background color, adds custom axes, plots data and labels the axes and title.

```python
import matplotlib.pyplot as plt

# Create a Figure with basic size and background color
fig = plt.figure(figsize=(6, 4), facecolor='lightblue')

# Add Axes to the Figure
ax = fig.add_axes([0.1, 0.1, 0.8, 0.8])
x = [1, 2, 3, 4]
y = [10, 20, 15, 25]
ax.plot(x, y)

plt.title("Simple Line Plot")
plt.xlabel("X-Axis")
plt.ylabel("Y-Axis")
plt.show()
```



Explanation:

- **fig.add_axes()** adds an Axes object to the figure.
- **[0.1, 0.1, 0.8, 0.8]** defines the position left, bottom, width and height as a fraction of figure size.

2. Axes Class

Axes class represents actual plotting area where data is drawn. It is the most basic and flexible for creating plots or subplots within a figure. A single figure can contain multiple axes but each Axes object belongs to only one figure. It can create an Axes object using axes() function.

Syntax: *axes([left, bottom, width, height])*

Example: This code creates a figure using Figure class and adds a custom axes area to it. It then plots two line graphs one for x vs y and another for y vs x. The graph includes axis labels, a title and a legend for better clarity and presentation.

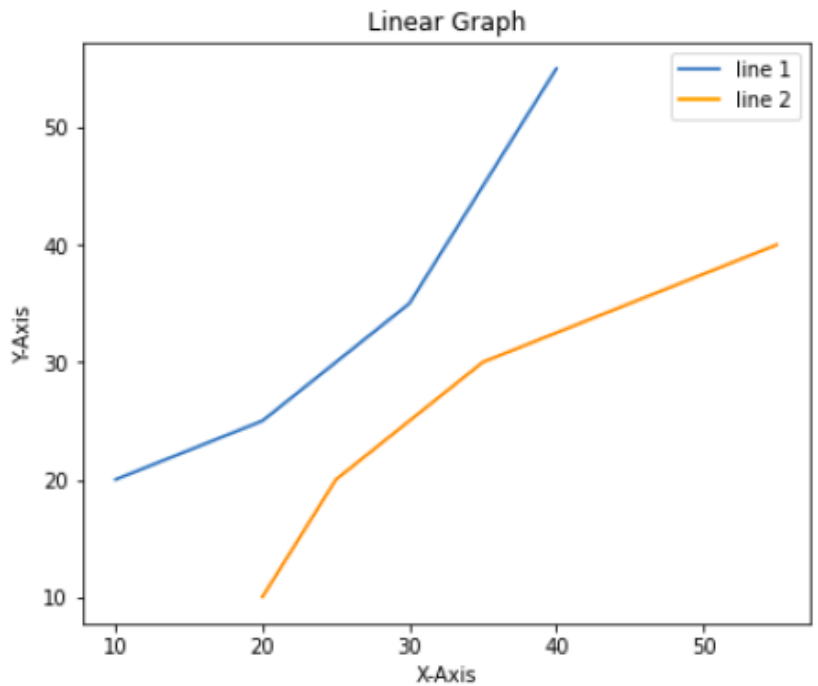
```
import matplotlib.pyplot as plt
from matplotlib.figure import Figure

x = [10, 20, 30, 40]
y = [20, 25, 35, 55]

fig = plt.figure(figsize=(5, 4))
ax = fig.add_axes([0.1, 0.1, 0.8, 0.8])

ax.plot(x, y)
ax.plot(y, x)

ax.set_title("Linear Graph")
ax.set_xlabel("X-Axis")
ax.set_ylabel("Y-Axis")
ax.legend(labels=('line 1', 'line 2'))
plt.show()
```



Advanced Techniques for Visualizing Subplots

1. Using add_axes()

add_axes() method allows us to manually add axes to a figure in Matplotlib. It takes a list of four values [left, bottom, width, height] to specify the position and size of the axes.

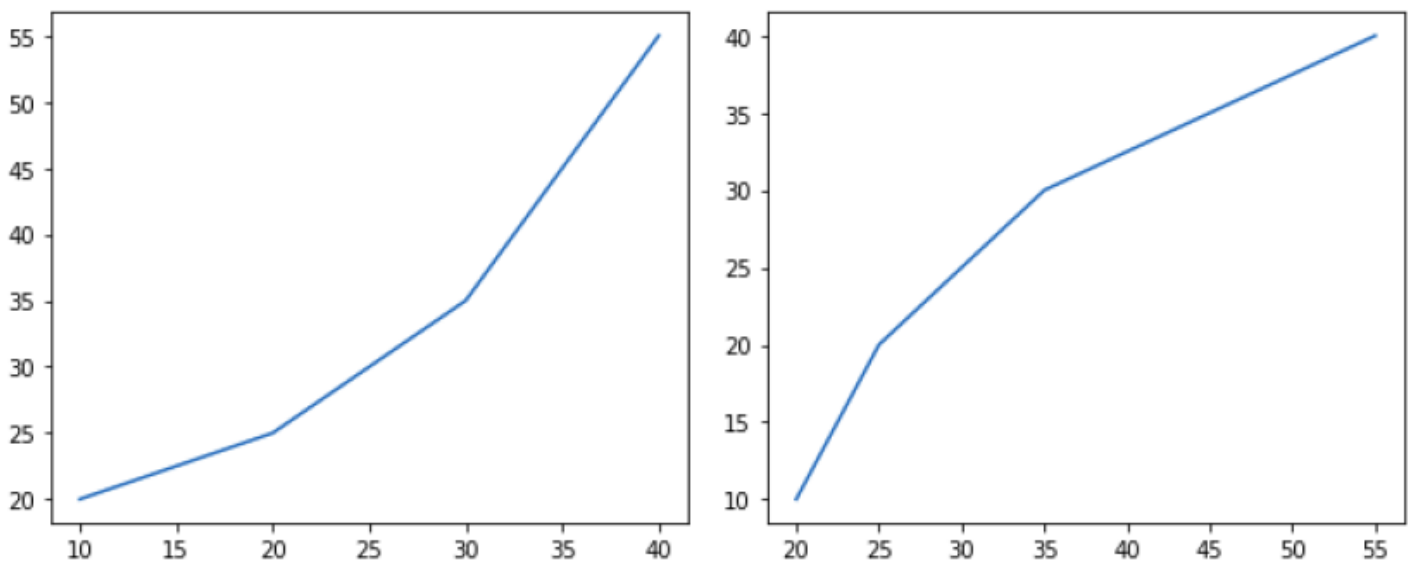
Example: This code creates two side-by-side plots using add_axes() one for x vs y and another for y vs x all in a single figure.

```
import matplotlib.pyplot as plt
from matplotlib.figure import Figure

x = [10, 20, 30, 40]
y = [20, 25, 35, 55]

fig = plt.figure(figsize=(10, 4))
ax1 = fig.add_axes([0.1, 0.1, 0.35, 0.8])
ax2 = fig.add_axes([0.55, 0.1, 0.35, 0.8])

ax1.plot(x, y)
ax2.plot(y, x)
plt.show()
```



2. Using subplot()

The subplot() method adds a plot to a specified grid position within the current figure. It takes three arguments: the number of rows, columns and plot index.

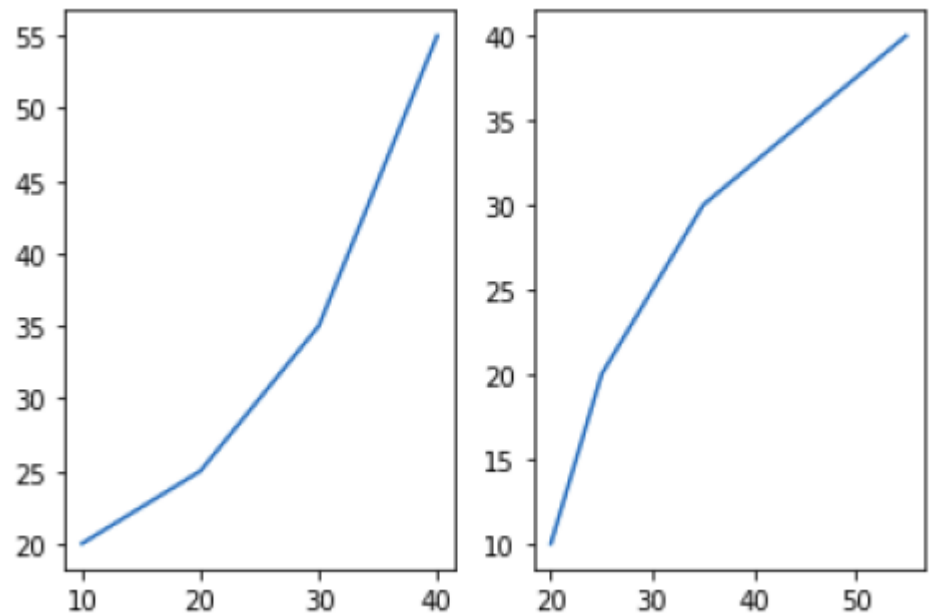
Example: This code uses subplot() to create two plots side by side one for x vs y and the other for y vs x within same figure.

```
import matplotlib.pyplot as plt

x = [10, 20, 30, 40]
y = [20, 25, 35, 55]
plt.figure()

plt.subplot(121)
plt.plot(x, y)

plt.subplot(122)
plt.plot(y, x)
plt.show()
```



3. Using subplot2grid()

The subplot2grid() creates axes object at a specified location inside a grid and also helps in spanning the axes object across multiple rows or columns.

Example: This code uses subplot2grid() to place two stacked plots in a single figure one on top of the other by defining their positions in a 7-row grid layout.

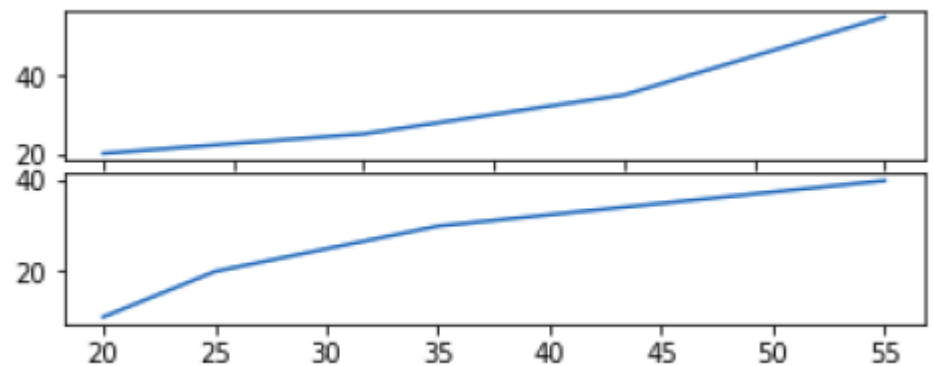
```
import matplotlib.pyplot as plt

x = [10, 20, 30, 40]
y = [20, 25, 35, 55]

axes1 = plt.subplot2grid (
(7, 1), (0, 0), rowspan = 2, colspan = 1)

axes2 = plt.subplot2grid (
(7, 1), (2, 0), rowspan = 2, colspan = 1)

axes1.plot(x, y)
axes2.plot(y, x)
plt.show()
```



Saving Plots Using savefig()

When plots are created using Matplotlib, users may want to save them as image files for use in reports, presentations or sharing. Matplotlib offers savefig() function to save the current plot to a file. By changing file extension, users can store the plot in different formats such as .png, .jpg, .pdf or .svg.

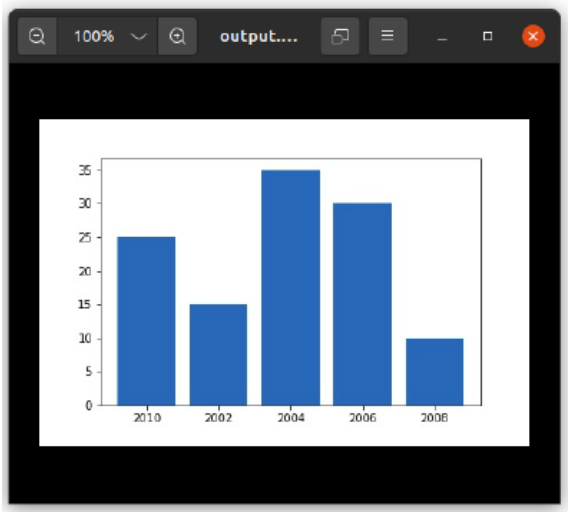
Example:

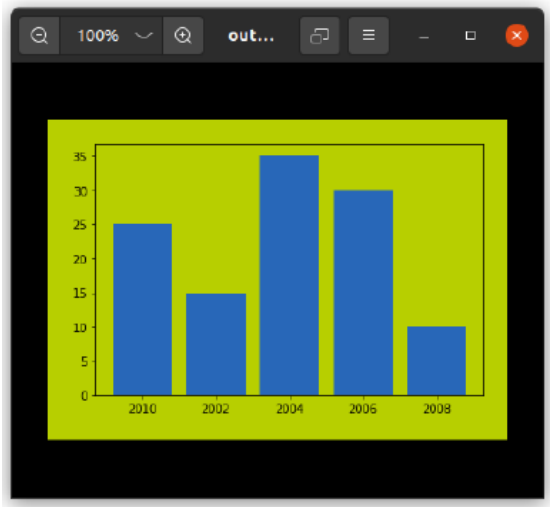
```
import matplotlib.pyplot as plt

year = ['2010', '2002', '2004', '2006', '2008']
production = [25, 15, 35, 30, 10]

plt.bar(year, production)

plt.savefig("output.jpg")
plt.savefig("output1", facecolor='y', bbox_inches="tight",pad_inches=0.3, transparent=True)
```





5 Seaborn - Python

Seaborn - Python

Seaborn is a Python library for creating attractive statistical visualizations. Built on Matplotlib and integrated with Pandas, it simplifies complex plots like line charts, heatmaps and violin plots with minimal code.

Creating Plots with Seaborn

Seaborn makes it easy to create clear and informative statistical plots with just a few lines of code. It offers built-in themes, color palettes, and functions tailored for different types of data.

Let's see various types of plots with simple code to understand how to use it effectively.

1. Line plot

A line plot shows the relationship between two numeric variables, often over time. It can also compare multiple groups using different lines.

Syntax: *sns.lineplot(x=None, y=None, data=None)*

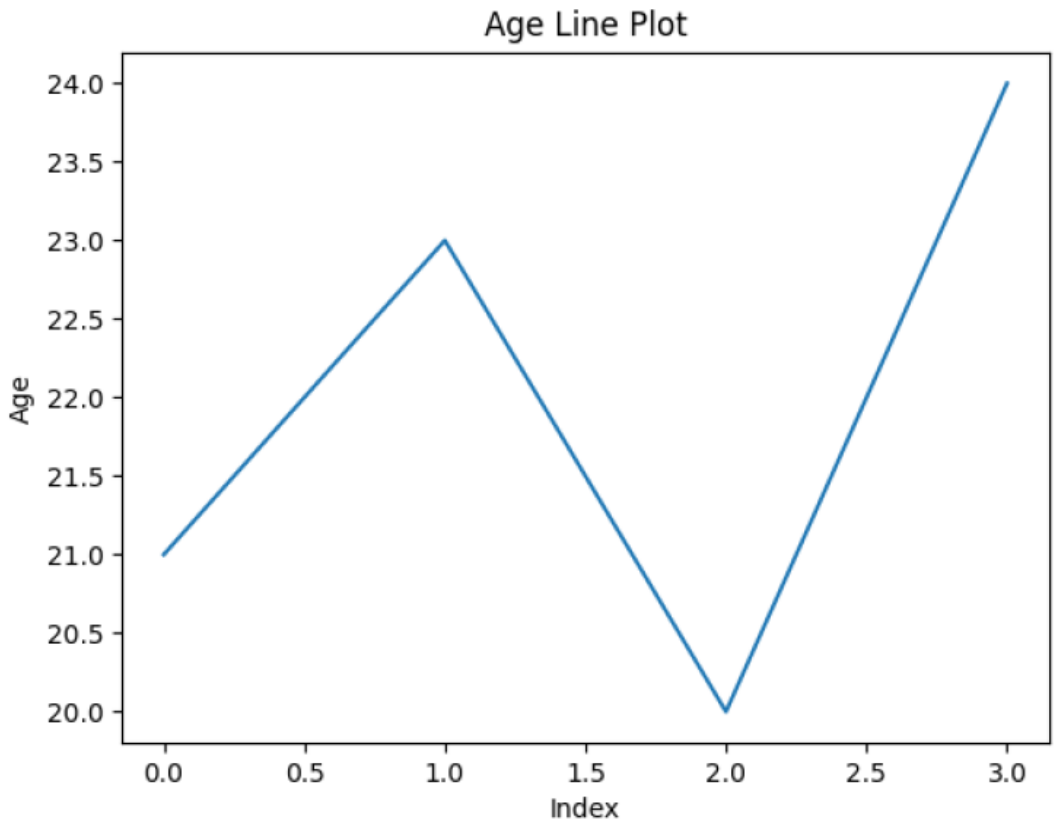
Parameters:

- **x, y**: Numeric input variables. These can be arrays, lists or column names from a DataFrame.
- **data**: DataFrame containing the data.

```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

data = {'Name': ['ANSH', 'SAHIL', 'JAYAN', 'ANURAG'], 'Age': [21, 23, 20, 24]}
df = pd.DataFrame(data)

sns.lineplot(x=df.index, y='Age', data=df)
plt.xlabel('Index')
plt.ylabel('Age')
plt.title('Age Line Plot')
plt.show()
```



2. Scatter Plot

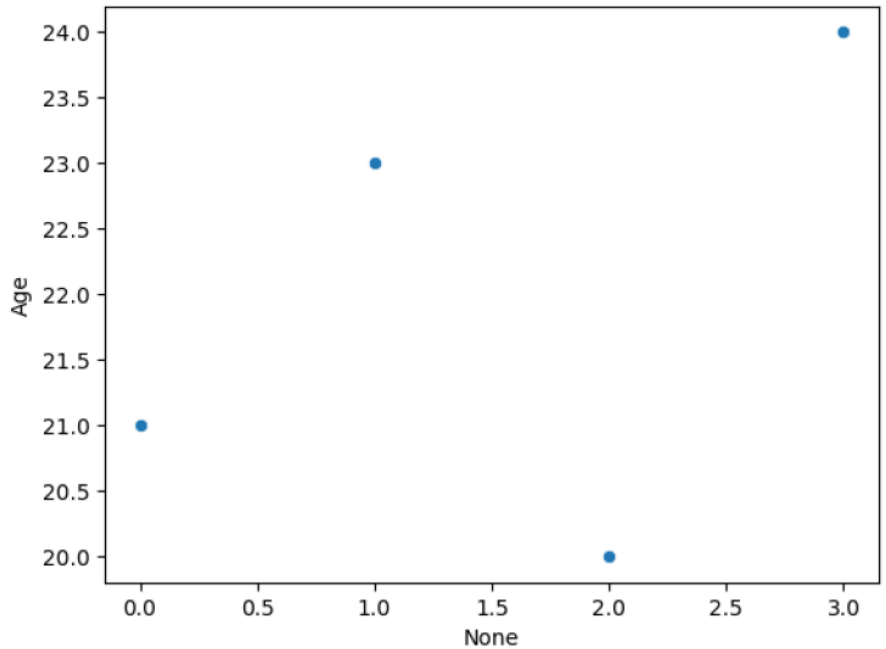
Scatter plots are used to visualize the relationship between two numerical variables. They help identify correlations or patterns. It can draw a two-dimensional graph.

Syntax: *sns.scatterplot(x=None, y=None, data=None)*

Parameters:

- **x, y:** Input data variables that should be numeric.
- **data (optional)***:* Dataset containing the variables.

Returns: An Axes object with the scatter plot.



Example:

```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

data = {'Name': ['ANSH', 'SAHIL', 'JAYAN', 'ANURAG'], 'Age': [21, 23, 20, 24]}
df = pd.DataFrame(data)

sns.scatterplot(x=df.index, y='Age', data=df)
plt.show()
```

3. Box plot

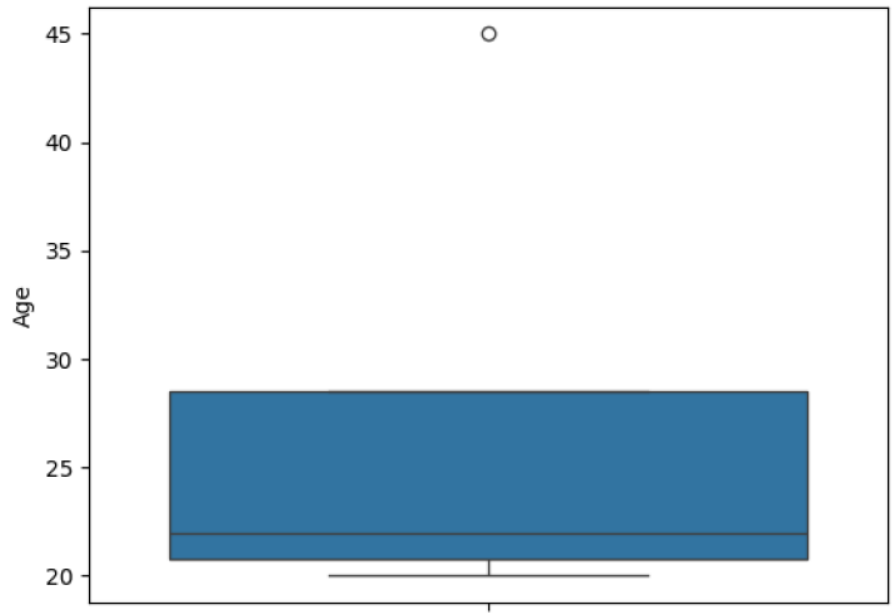
A box plot is the visual representation of the depicting groups of numerical data with their quartiles against continuous/categorical data. It consists of 5 key statistics: Minimum ,First Quartile or 25% , Median (Second Quartile) or 50%, Third Quartile or 75% and Maximum

Syntax: *sns.boxplot(x=None, y=None, hue=None, data=None)*

Parameters:

- **x, y, hue:** Variables for plotting long-form data.
- **data:** Dataset to plot. If x and y are absent data is treated as wide-form.

Returns: An Axes object with the box plot.



Example:

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

data = {'Name': ['ANSH', 'SAHIL', 'JAYAN', 'ANURAG'], 'Age': [21, 23, 20, 45]}
df = pd.DataFrame(data)

sns.boxplot(y='Age', data=df)
plt.show()
```

4. Violin Plot

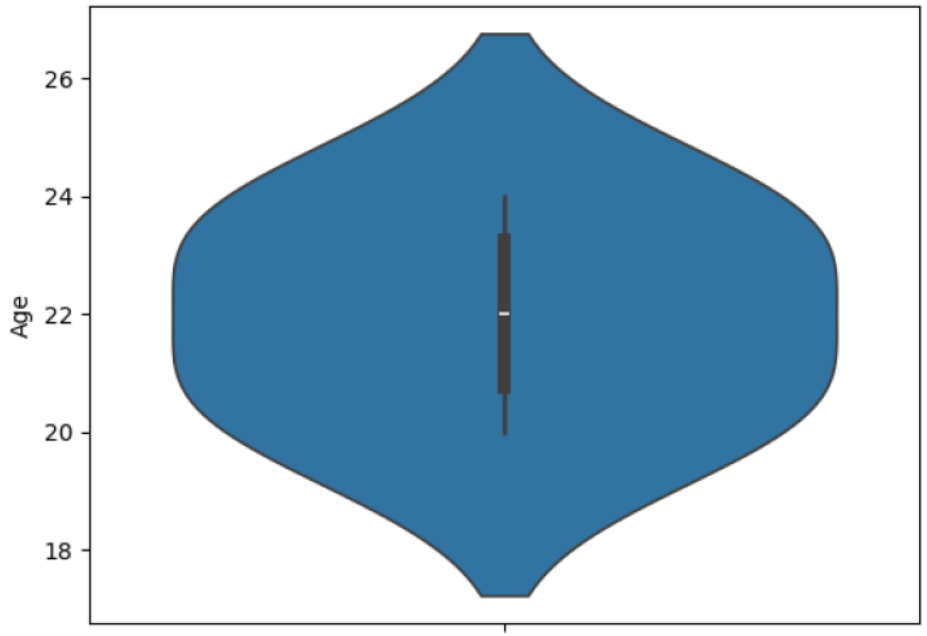
A violin plot is similar to a boxplot. It shows several quantitative data across one or more categorical variables such that those distributions can be compared.

Syntax: *sns.violinplot(x=None, y=None, hue=None, data=None)*

Parameters:

- **x, y, hue:** Inputs for plotting long-form data.
- **data:** Dataset for plotting.

Example:




```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

data = {'Name': ['ANSH', 'SAHIL', 'JAYAN', 'ANURAG'], 'Age': [21, 23, 20, 24]}
df = pd.DataFrame(data)

sns.violinplot(y='Age', data=df)
plt.show()
```

5. Swarm plot

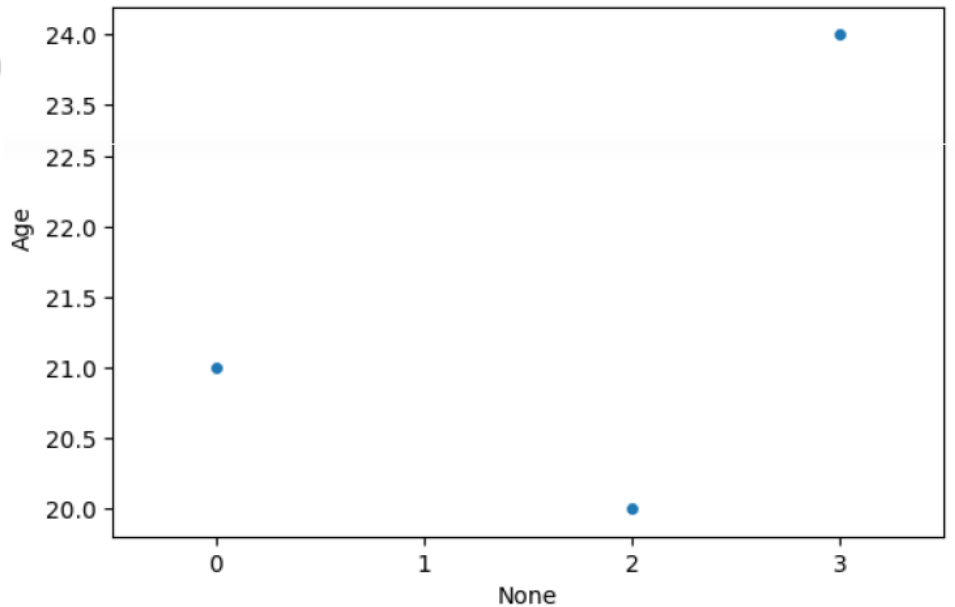
A swarm plot displays individual data points without overlap along a categorical axis which provides a clear view of distribution density.

Syntax: *sns.swarmplot(x=None, y=None, hue=None, data=None)*

Parameters:

- **x, y, hue:** Inputs for plotting long-form data.
- **data:** Dataset for plotting.

Example:




```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

data = {'Name': ['ANSH', 'SAHIL', 'JAYAN', 'ANURAG'], 'Age': [21, 23, 20, 24]}
df = pd.DataFrame(data)

sns.swarmplot(x=df.index, y='Age', data=df)
plt.show()
```

6. Bar plot

Barplot represents an estimate of central tendency for a numeric variable with the height of each rectangle and provides some indication of the uncertainty around that estimate using error bars.

Syntax: *sns.barplot(x=None, y=None, hue=None, data=None)*

Parameters :

- **x, y :** Variables or column names for long-form data.
- **hue :** (optional) Column for color encoding.
- **data :** (optional) Dataset to plot.

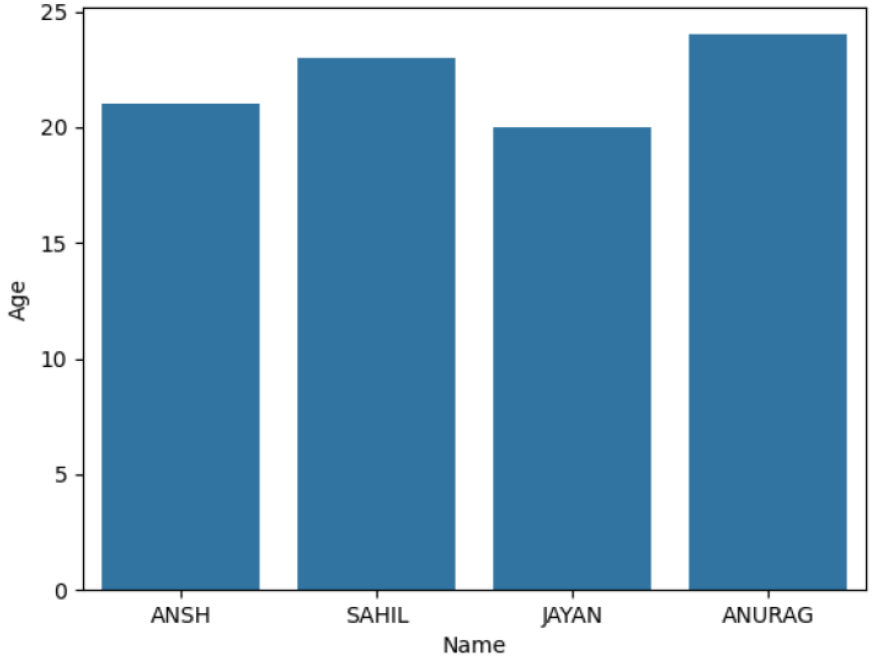
Returns: Axes object with the bar plot.

Example:

```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

data = {'Name': ['ANSH', 'SAHIL', 'JAYAN', 'ANURAG'], 'Age': [21, 23, 20, 24]}
df = pd.DataFrame(data)

sns.barplot(x='Name', y='Age', data=df)
plt.show()
```



7. Point plot

Point plot show point estimates and confidence intervals using scatter glyphs which represents the central tendency of a numeric variable.

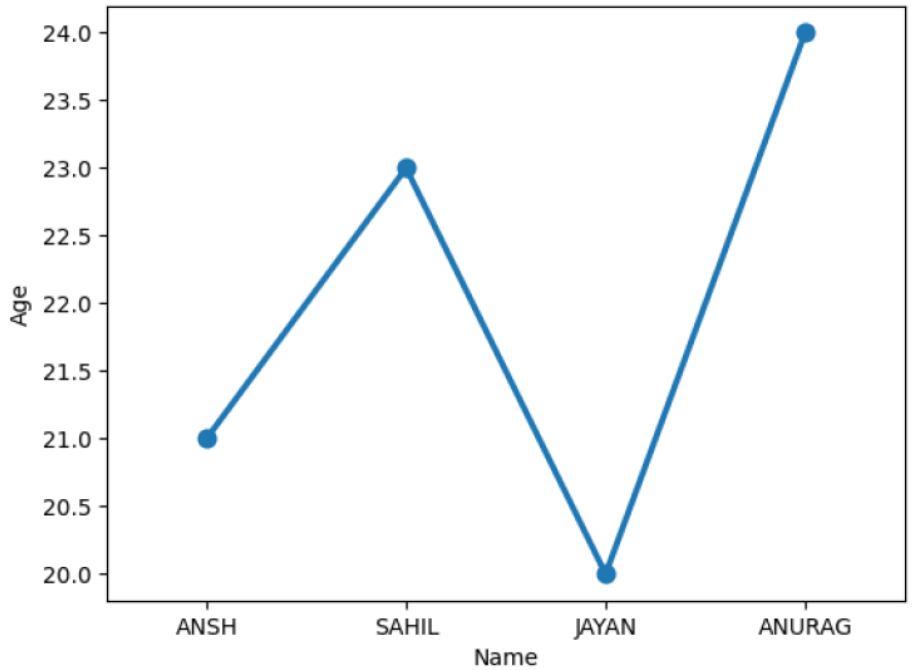
Syntax: *sns.pointplot(x=None, y=None, hue=None, data=None)*

Parameters:

- **x, y:** Inputs for plotting long-form data.
- **hue:** (optional) column name for color encoding.
- **data:** Dataframe as a Dataset for plotting.

Return: Axes object with the point plot.

Example:




```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

data = {'Name': ['ANSH', 'SAHIL', 'JAYAN', 'ANURAG'],'Age': [21, 23, 20, 24]}
df = pd.DataFrame(data)

sns.pointplot(x='Name', y='Age', data=df)
plt.show()
```

8. Count plot

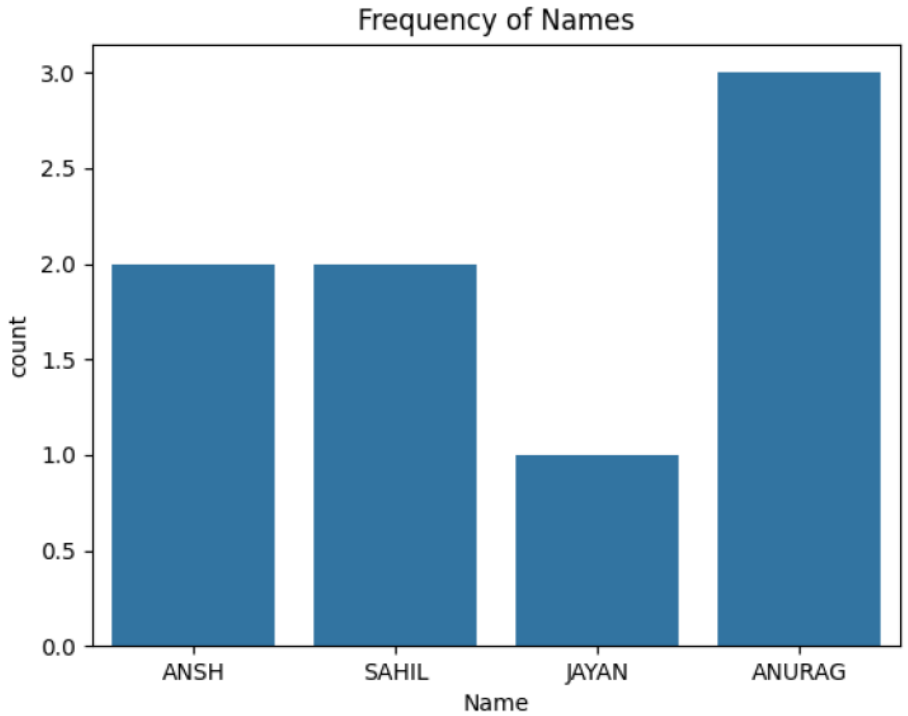
A Count plot displays the number of occurrences of each category using bars to visualize the distribution of categorical variables.

Syntax : *sns.countplot(x=None, y=None, hue=None, data=None)*

Parameters :

- **x, y:** Inputs for plotting long-form data.
- **hue:** (optional) column name for color encoding.
- **data:** Dataframe as a Dataset for plotting.

Returns: Axes object with the count plot.



Example:

```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

data = {'Name': ['ANSH', 'SAHIL', 'ANSH', 'JAYAN', 'ANURAG', 'ANURAG', 'ANURAG', 'SAHIL']}
df = pd.DataFrame(data)

sns.countplot(x='Name', data=df)
plt.title("Frequency of Names")
plt.show()
```

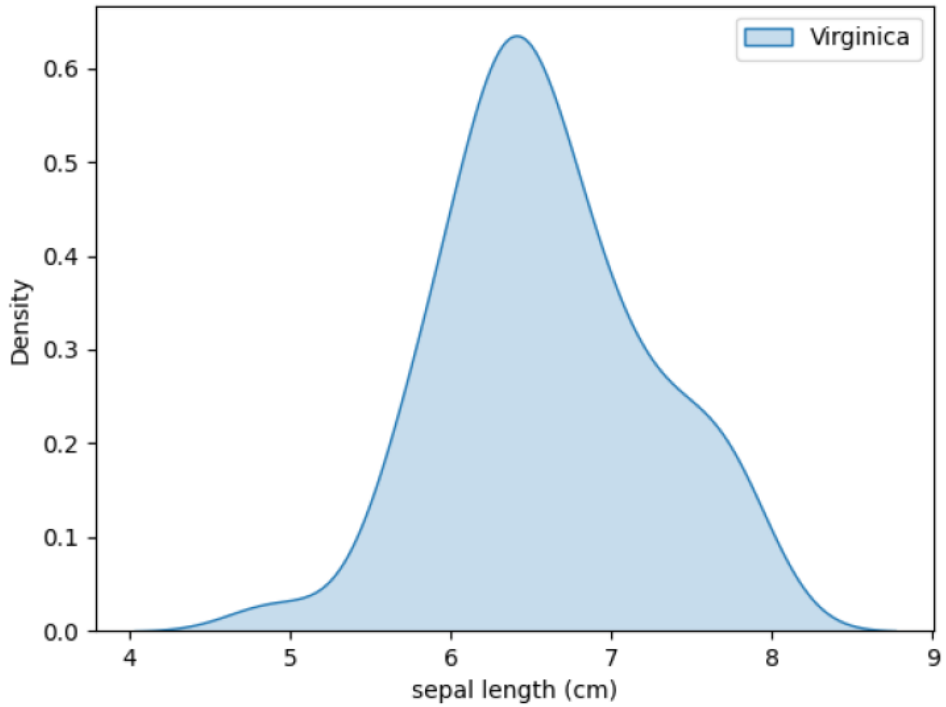
9. KDE Plot

KDE Plot (**Kernel Density Estimate)** is used for visualizing the Probability Density of a continuous variable at different values in a continuous variable. We can also plot a single graph for multiple samples which helps in more efficient data visualization.

Syntax: *sns.kdeplot(data=None, x=None, y=None, fill=False)*

Parameters:

- **x, y:** Vectors or data keys.
- **vertical:** Boolean to plot vertically.
- **palette:** Color palette.
- **data:** Dataframe



Example:

```
from sklearn.datasets import load_iris
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

iris = load_iris()
df = pd.DataFrame(iris.data, columns=iris.feature_names)
df['Species'] = iris.target
df['Species'] = df['Species'].map({ 0: 'Setosa', 1: 'Versicolor', 2: 'Virginica'})

sns.kdeplot(data=df[df['Species'] == 'Virginica'], x='sepal length (cm)', fill=True, label='Virginica')
plt.legend()
plt.show()
```

How to Customize Seaborn Plots with Python?

Customizing Seaborn plots increases their readability and visual appeal which makes the data insights clearer and more informative. Here are several ways we can customize our plots in Seaborn:

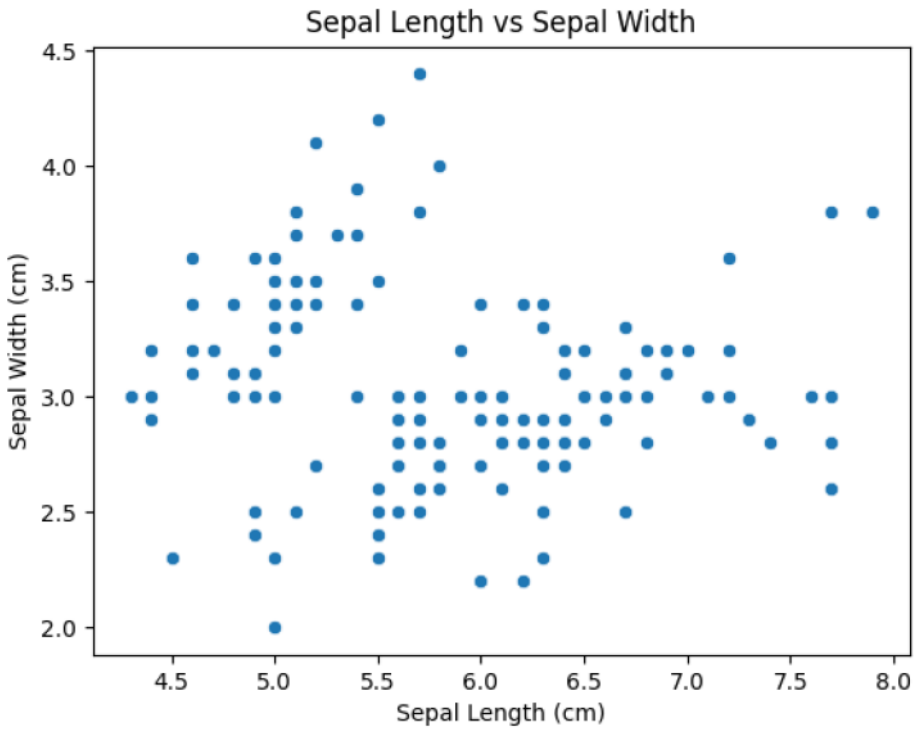
1. Adding Titles and Axis Labels

Adding descriptive titles and axis labels makes our plots more understandable and informative. Using Matplotlib's **plt.title()**, **plt.xlabel()** and **plt.ylabel()** to set titles and axis labels.

```
import seaborn as sns
import matplotlib.pyplot as plt

iris = sns.load_dataset('iris')
sns.scatterplot(x='sepal_length', y='sepal_width', data=iris)

# Add plot title and axis labels
plt.title('Sepal Length vs Sepal Width')
plt.xlabel('Sepal Length (cm)')
plt.ylabel('Sepal Width (cm)')
plt.show()
```

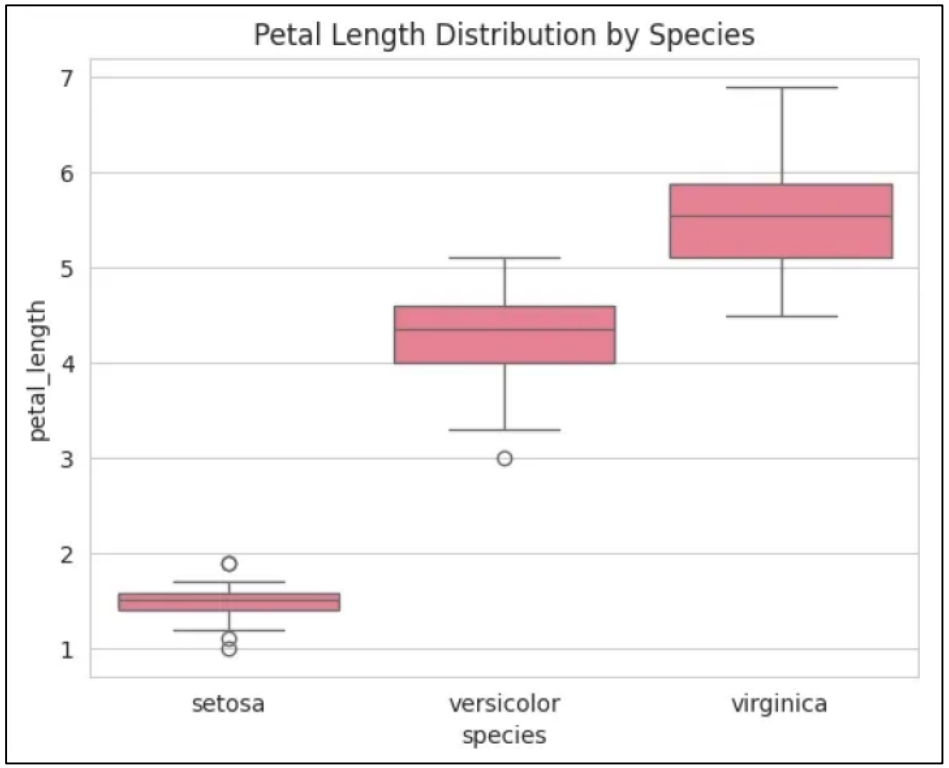


2. Built-in Styles and Grids in Seaborn

Seaborn provides built-in styles that control the background and grid of your plots. These styles improve readability and can be chosen based on your presentation needs.

Available Styles:

- darkgrid – Dark background with light gridlines.
  Great for clear contrast.
- whitegrid – White background with light gridlines.
  Ideal for statistical plots.
- dark – Dark background without gridlines.
  Clean and modern look.
- white – Plain white background without gridlines.
  Good for simple visuals.
- ticks – White background with axis ticks styled sharply.
  Suitable for publications.




```
import seaborn as sns
import matplotlib.pyplot as plt

sns.set_style("whitegrid")

sns.boxplot(x='species', y='petal_length', data=sns.load_dataset('iris'))
plt.title('Petal Length Distribution by Species')
plt.show()
```

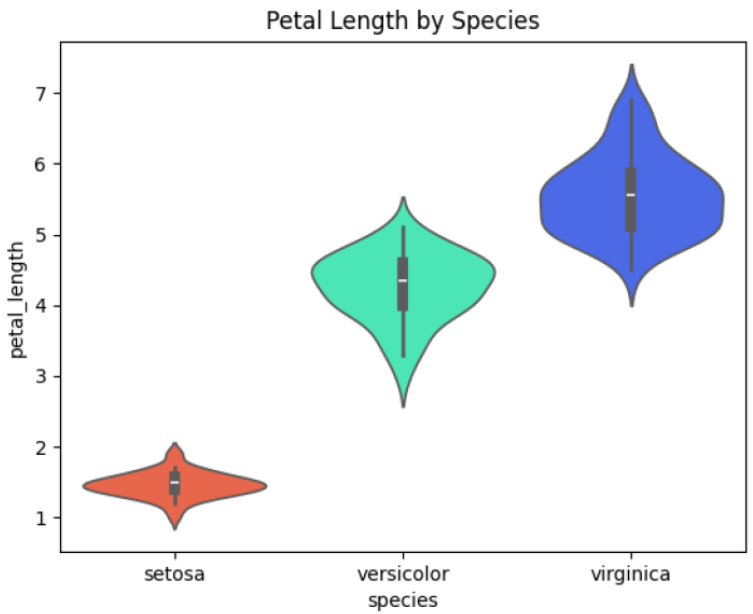
3. Customizing Color Palettes

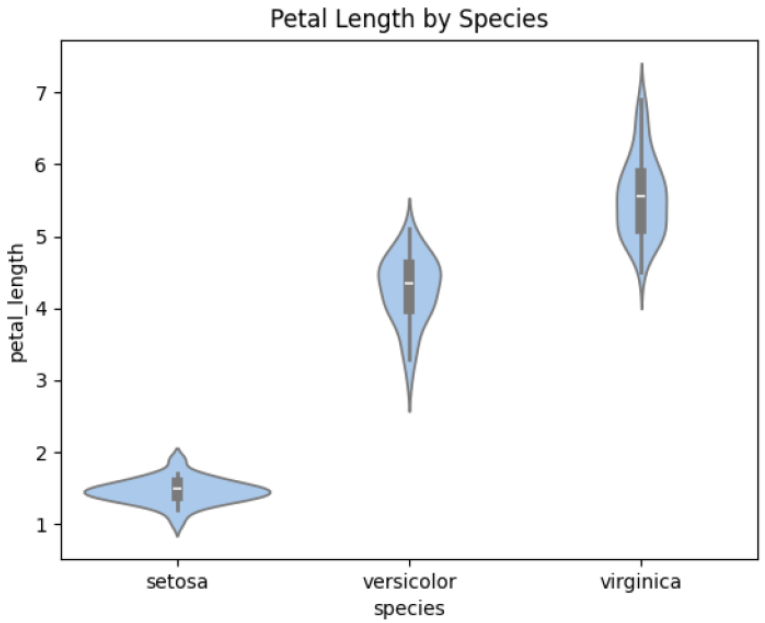
Seaborn makes it easy to enhance the appearance of plots using color palettes. You can choose from built-in palettes like "deep", "muted", or "bright" or define your own using sns.color_palette(). Customizing colors improves clarity and helps match your data's theme or purpose.

a) Using a Built-in Palette:

```
sns.set_palette("pastel")

sns.violinplot(x='species', y='petal_length', data=sns.load_dataset('iris'))
plt.title('Petal Length Distribution by Species')
plt.show()
```





b) Using a Custom Palette:

```
custom_colors = ['#FF5733', '#33FFBD', '#335BFF']
sns.set_palette(custom_colors)

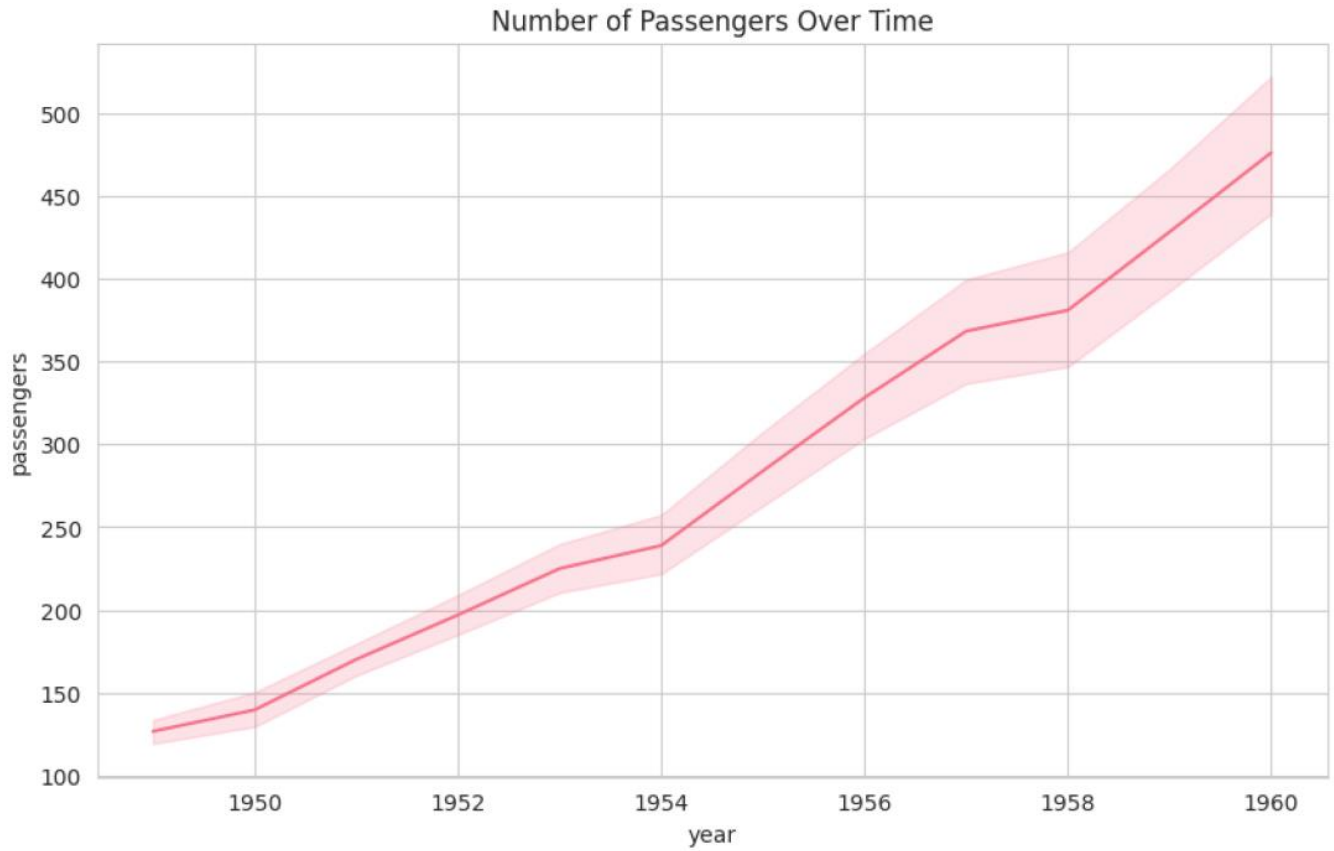
sns.violinplot(x='species', y='petal_length', data=sns.load_dataset('iris'))
plt.title('Custom Colored Petal Length Distribution')
plt.show()
```

4. Adjusting Figure Size and Aspect Ratio

We can adjust the figure size using **plt.figure(figsize=(width,height))** to control the plot's dimensions. This allows for better customization to fit different presentation or reports.

```
plt.figure(figsize=(10, 6))

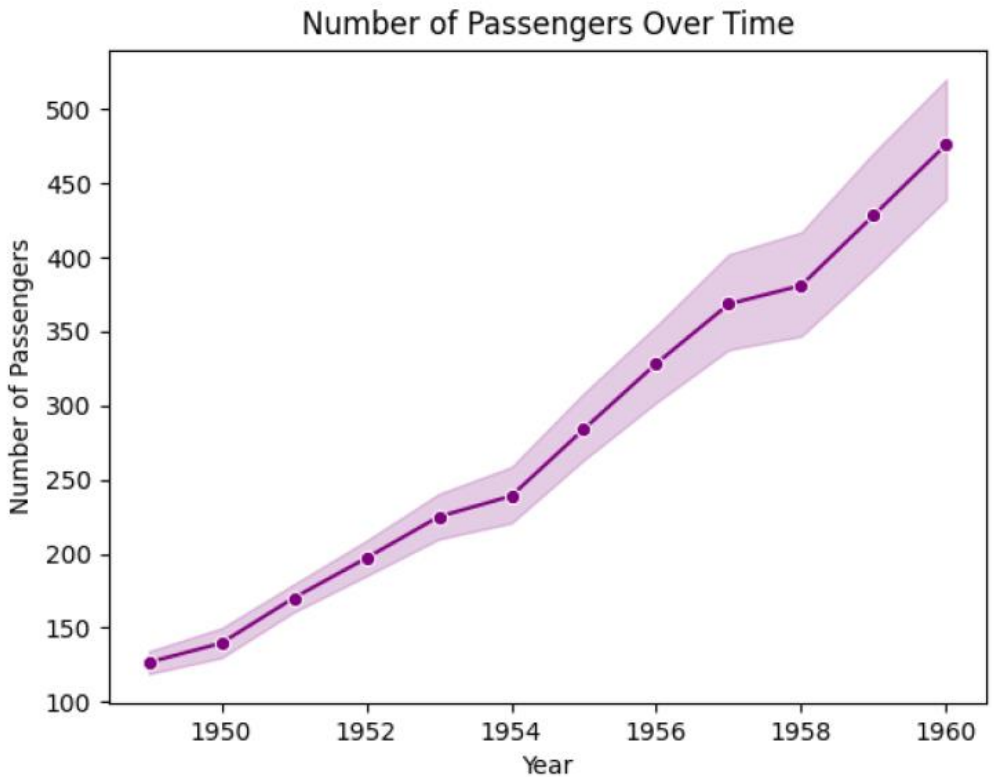
sns.lineplot(x='year', y='passengers', data=sns.load_dataset('flights'))
plt.title('Number of Passengers Over Time')
plt.show()
```



5. Adding Markers to Line Plots

Markers can be added to Seaborn line plots using the marker argument to highlight data points. For example adding circular markers to the line plot using sns.lineplot(x='x', y='y' ,marker='o')

```
sns.lineplot(x='year', y='passengers', data=sns.load_dataset('flights'), marker='o')
plt.title('Number of Passengers Over Time')
plt.show()
```



Visualizing Relationships and Patterns with Seaborn

These visualizations help to find hidden patterns and correlations in datasets with multiple variables.

1. Pair Plots

Pair plots are used explore relationships between several variables by generating scatter plots for every pair of variables in a dataset along with univariate distributions on the diagonal. This is useful for exploring datasets with multiple variables and seeing potential correlations.

Syntax: *sns.pairplot(data, hue=None)*

Parameters:
- **data:** Dataset to plot.
- **hue:** (optional) Categorical variable used for color coding data points.

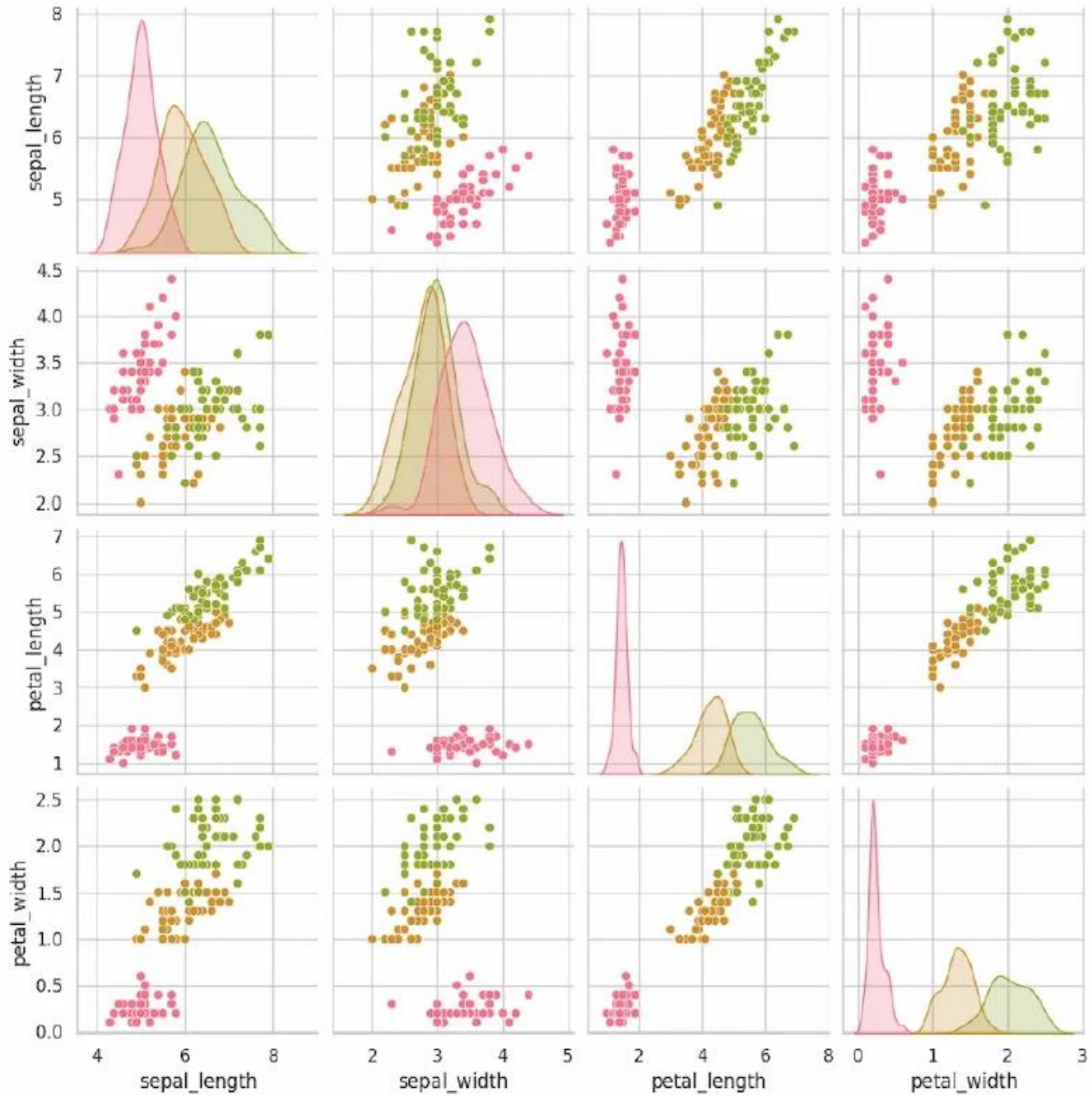
Returns: An array of Axes objects containing the scatter plot grid and distributions.

Example:

```python
import seaborn as sns
import matplotlib.pyplot as plt

sns.set_style("whitegrid")
custom_palette = sns.color_palette("husl", 8)
sns.set_palette(custom_palette)

data = sns.load_dataset("iris")
sns.pairplot(data, hue="species")
plt.show()
```



2. Joint Plots

Joint plots combine a scatter plot with the distributions of the individual variables. This allows for a quick visual representation of how the variables are distributed individually and how they relate to one another.

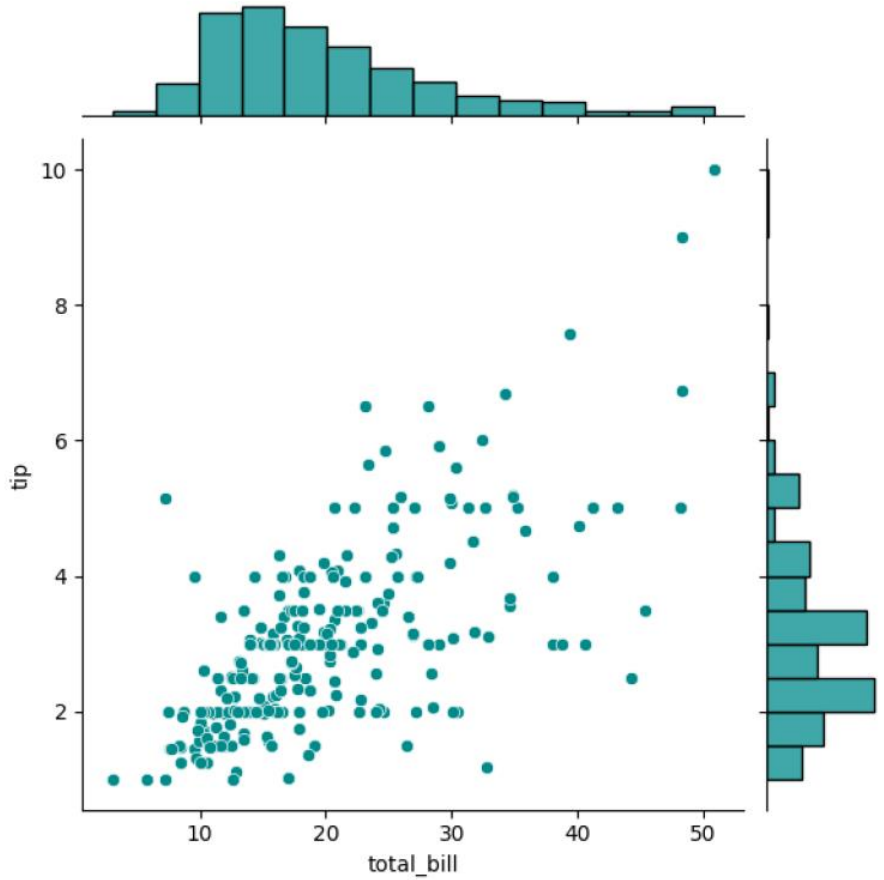
Syntax: *sns.jointplot(x, y, data, kind='scatter')*

Parameters:

- **x, y:** Variables to plot.
- **data:** Dataset to plot.
- **kind:** Type of plot to display ('scatter', 'kde', 'reg' etc).

Returns: An Axes object with the joint plot including scatter plot and distribution plots on the margins.

Example:




```
import seaborn as sns
import matplotlib.pyplot as plt

data = sns.load_dataset("tips")
sns.jointplot(x="total_bill", y="tip", data=data, kind="scatter", color="#008B8B")
plt.show()
```

3. Grid Plot

Grid plots in Seaborn are used to create multiple subplots in a grid layout. Using Seaborn's FacetGrid we can visualize how variables interact across different categories which makes it easier to compare groups or conditions within our dataset.

Syntax: *g = sns.FacetGrid(data, col='column_name', row='row_name')*
g.map(sns.scatterplot, 'x', 'y')

Parameters:

- **data**: Dataset to plot.
- **col, row**: Variables for the columns and rows of the grid (categorical variables).
- **sns.scatterplot**: The plotting function to apply to each facet.

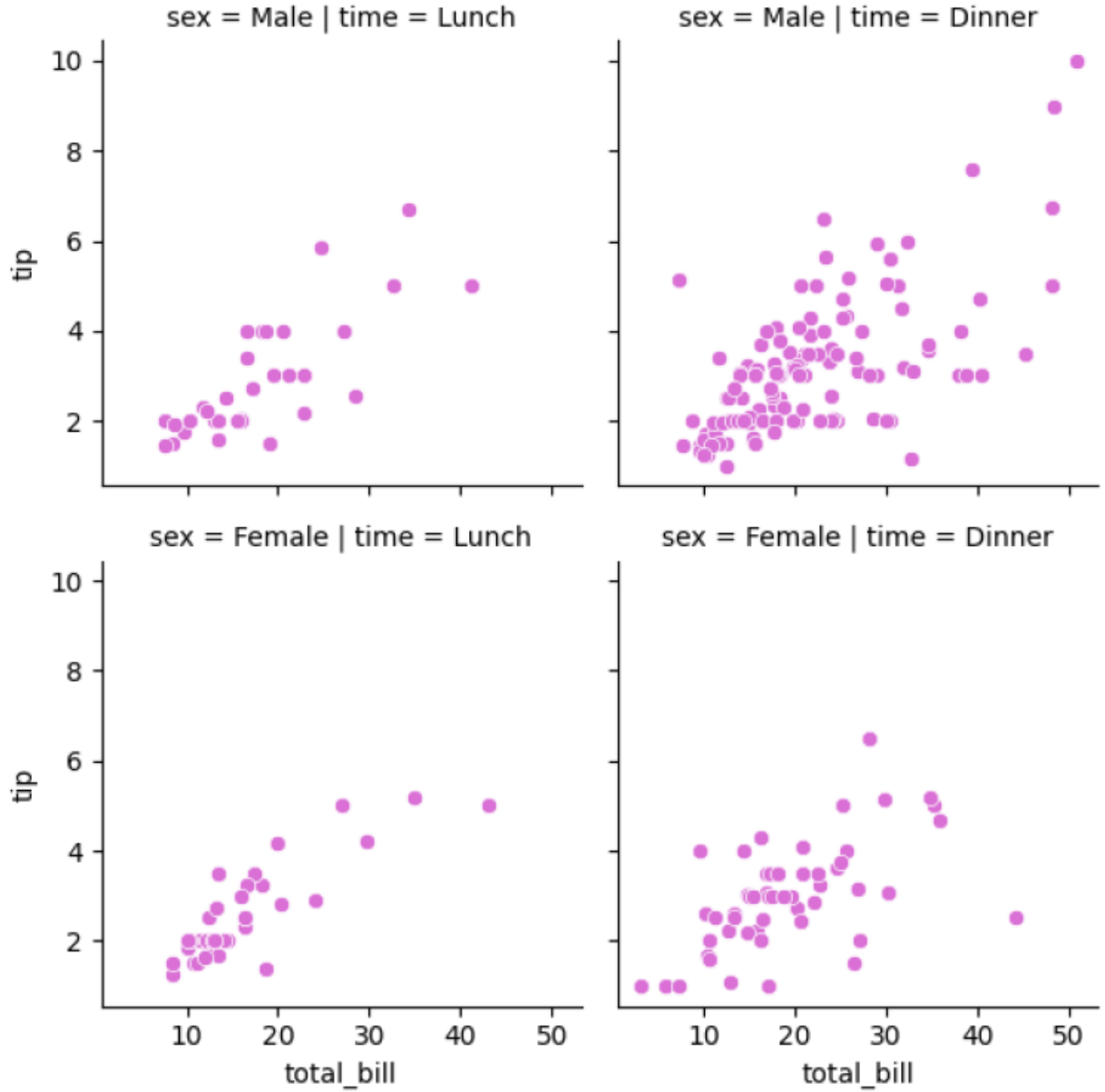
Returns: A **FacetGrid** object with the grid of plots.

Example: To use FacetGrid, we first need to initialize it with a dataset and specify the variables that will form the row, column or hue dimensions of the grid. Here is an example using the tips dataset:

```
import seaborn as sns
import matplotlib.pyplot as plt

tips = sns.load_dataset("tips")

plot=sns.FacetGrid(tips, col="time", row="sex")
plot.map(sns.scatterplot, "total_bill", "tip")
plt.show()
```



Regression Plots: Visualizing Linear Relationships

Seaborn simplifies the process of performing and visualizing regressions specifically linear regressions which is important for identifying relationships between variables, detecting trends and making predictions. It supports two primary functions for regression visualization:

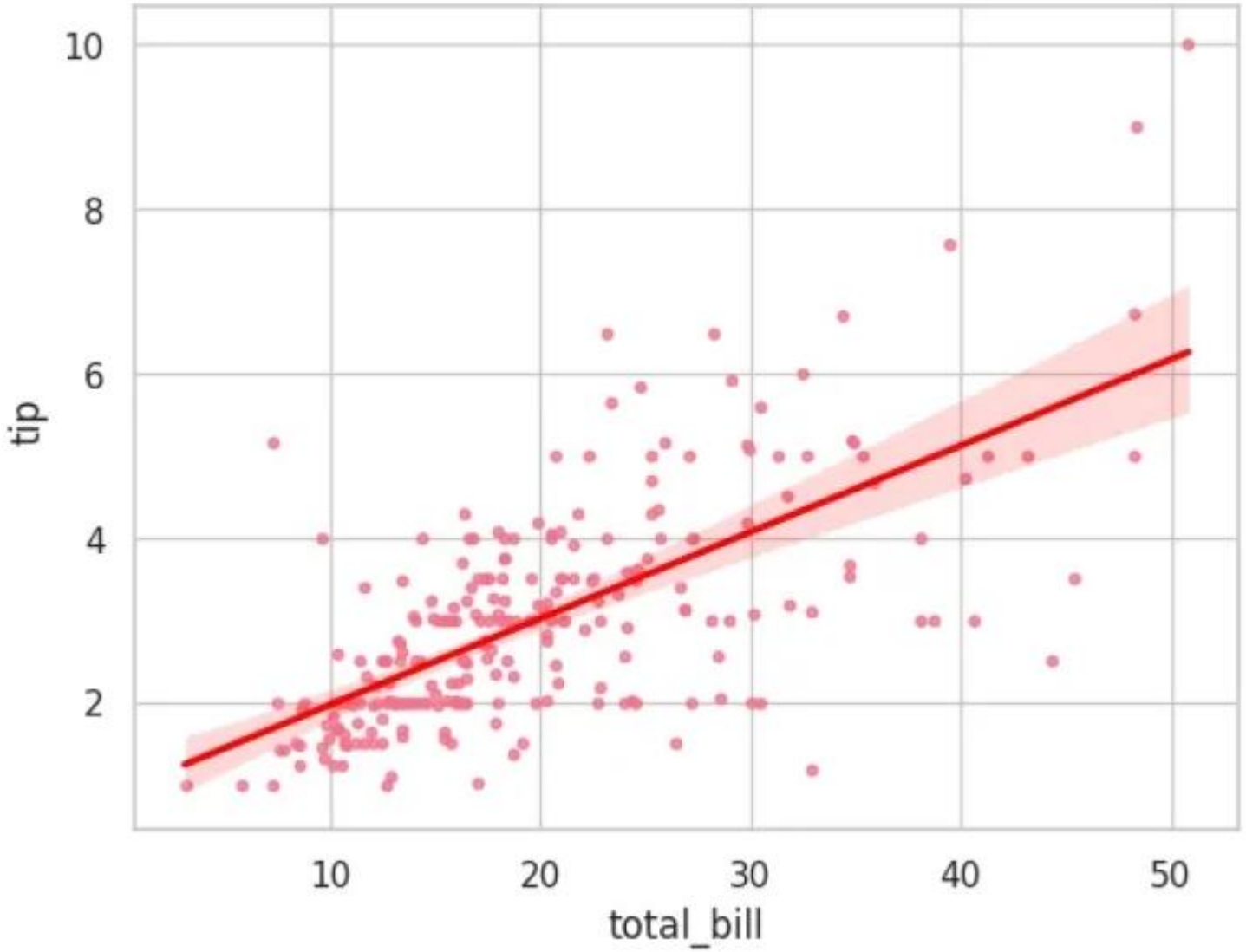
- regplot(): This function plots a scatter plot along with a linear regression model fit.
- lmplot(): This function also plots linear models but provides more flexibility in handling multiple facets and datasets.

Example: Let's use a simple dataset to visualize a linear regression between two variables: `x` (independent variable) and `y` (dependent variable).

```
import seaborn as sns
import matplotlib.pyplot as plt

tips = sns.load_dataset('tips')

sns.regplot(x='total_bill', y='tip', data=tips, scatter_kws={'s':10}, line_kws={'color':'red'})
plt.show()
```

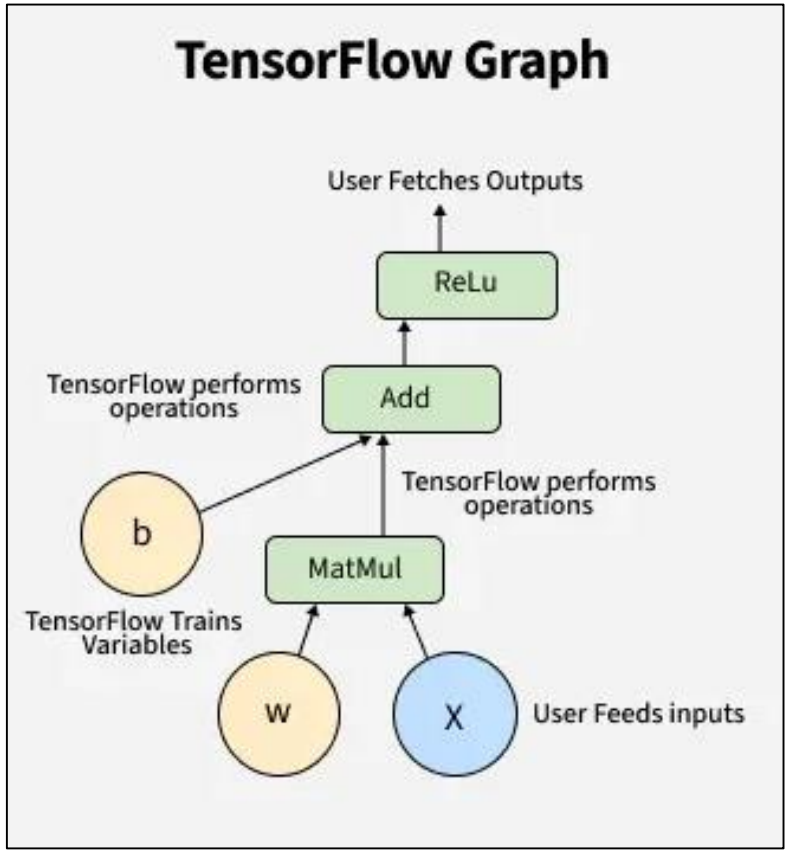

10
8
6
4
2
tip
10
20
30
40
50
total_bill

6 TensorFlow

TensorFlow

TensorFlow is an open-source framework for machine learning and artificial intelligence developed by Google Brain. It provides tools to build, train and deploy models across different platforms, especially for deep learning tasks.

- Supports a wide range of applications such as NLP, computer vision, time series forecasting and reinforcement learning
- Enables scalable model development and deployment across devices



Key Features

1. Scalability

TensorFlow is designed to scale across a variety of platforms from desktops and servers to mobile devices and embedded systems. It supports distributed computing allowing models to be trained on large datasets efficiently.

2. Comprehensive Ecosystem

TensorFlow offers a broad set of tools and libraries including:

- **TensorFlow Core:** The base API for TensorFlow that allows users to define models, build computations and execute them.
- **Keras:** A high-level API for building neural networks that runs on top of TensorFlow, simplifying model development.
- **TensorFlow Lite:** A lightweight solution for deploying models on mobile and embedded devices.
- **TensorFlow.js:** A library for running machine learning models directly in the browser using JavaScript.
- **TensorFlow Extended (TFX):** A production-ready solution for deploying machine learning models in production environments.
- **TensorFlow Hub:** A repository of pre-trained models that can be easily integrated into applications.

3. Automatic Differentiation (Autograd)

TensorFlow automatically calculates gradients for all trainable variables in the model which simplifies the backpropagation process during training. This is a core feature that enables efficient model optimization using techniques like gradient descent.

4. Multi-language Support

TensorFlow is primarily designed for Python but it also provides APIs for other languages like C++, Java and JavaScript making it accessible to developers with different programming backgrounds.

5. TensorFlow Serving and TensorFlow Model Optimization

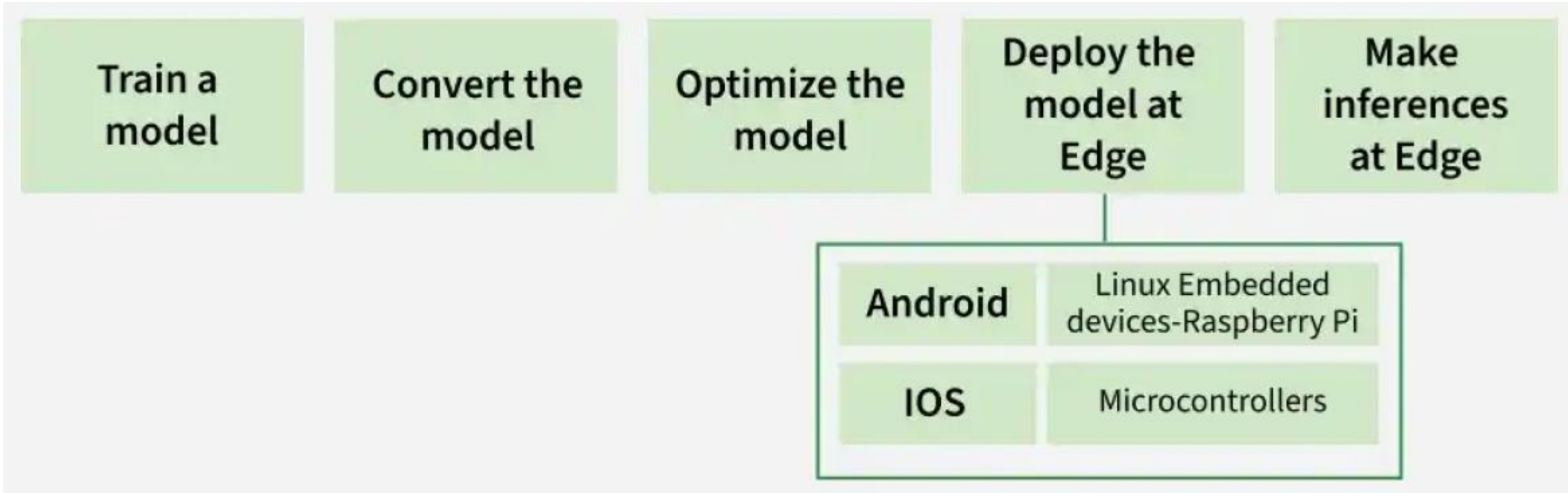
TensorFlow includes tools for serving machine learning models in production environments and optimizing them for inference allowing for lower latency and higher efficiency.

TensorFlow Architecture

The architecture of TensorFlow revolves around the concept of a computational graph which is a network of nodes (operations) and edges (data). Here's a breakdown of key components:

- **Tensors:** Tensors are the fundamental units of data in TensorFlow. They are multi-dimensional arrays or matrices used for storing data. A tensor can have one dimension (vector), two dimensions (matrix) or more dimensions.
- **Graph:** A TensorFlow graph represents a computation as a flow of tensors through a series of operations. Each operation in the graph performs a specific mathematical function on the input tensors such as matrix multiplication, addition or activation.
- In TensorFlow 2.x, computations are executed eagerly by default, meaning operations run immediately without requiring a separate session.

TensorFlow Workflow



Building a machine learning model in TensorFlow typically involves the following steps:

Step 1: Train a Model

- Use TensorFlow to build and train a machine learning model on platform like a PC or cloud.
- Employ datasets relevant to our application like images, text, sensor data, etc.
- Evaluate and validate the model to ensure high accuracy before deployment.

Step 2: Convert the Model

- Convert the trained model into TensorFlow Lite (.tflite) format using the TFLite Converter.
- This conversion prepares the model for resource-constrained edge environments.
- Supports different formats like saved models, Keras models or concrete functions.

Step 3: Optimize the Model

- Apply model optimization techniques such as quantization, pruning or weight clustering.
- Reduces the model size, improves inference speed and minimizes memory footprint.
- Crucial for running models efficiently on mobile, embedded or microcontroller devices.

Step 4: Deploy the Model

- Deploy the optimized .tflite model to edge devices like Android, iOS, Linux-based embedded systems like Raspberry Pi and Microcontrollers like Arm Cortex-M.
- Ensure compatibility with TensorFlow Lite runtime for the target platform.

Step 5: Make Inferences at the Edge

- Run real-time predictions directly on the edge device using the TFLite Interpreter.
- Enables low-latency, offline inference without relying on cloud computation.
- Supports use cases like image recognition, voice detection and sensor data analysis.

Building a Simple Model with TensorFlow

1. Here, we have loaded the MNIST Dataset and process the image. Then we have built a simple neural network using TensorFlow's Sequential API with two layers:

   - Dense layer with ReLU activation
   - An output layer with softmax activation function

2. At last we compiled the model using Adam Optimizer and Sparse Categorical Crossentropy and train the model for 5 epochs.

3. Install tensorfow library by running the following command in command prompt

   pip install tensorflow

4. Implementation

```
from tensorflow.keras.datasets import mnist
from tensorflow.keras.layers import Dense
from tensorflow.keras.models import Sequential
import tensorflow as tf

(train_images, train_labels), (test_images, test_labels) = mnist.load_data()

train_images = train_images.reshape(
    (train_images.shape[0], 28 * 28)).astype('float32') / 255
test_images = test_images.reshape(
    (test_images.shape[0], 28 * 28)).astype('float32') / 255

model = Sequential([
    Dense(128, activation='relu', input_shape=(28 * 28,)),
    Dense(10, activation='softmax')
])

model.compile(optimizer='adam',
              loss='sparse_categorical_crossentropy', metrics=['accuracy'])

model.fit(train_images, train_labels, epochs=5)

test_loss, test_acc = model.evaluate(test_images, test_labels)
print(f"Test accuracy: {test_acc}")
```

Output:

```
Epoch 1/5

1875/1875 ━━━━━━━━━━━━━━━━━━━━ 7s 3ms/step - accuracy: 0.8771 - loss: 0.4387
Epoch 2/5
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 11s 3ms/step - accuracy: 0.9632 - loss: 0.1214
Epoch 3/5
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 5s 3ms/step - accuracy: 0.9758 - loss: 0.0792
Epoch 4/5
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 7s 3ms/step - accuracy: 0.9829 - loss: 0.0563
Epoch 5/5
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 5s 3ms/step - accuracy: 0.9869 - loss: 0.0442
313/313 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.9727 - loss: 0.0869

Test accuracy: 0.9769999980926514
```

Training

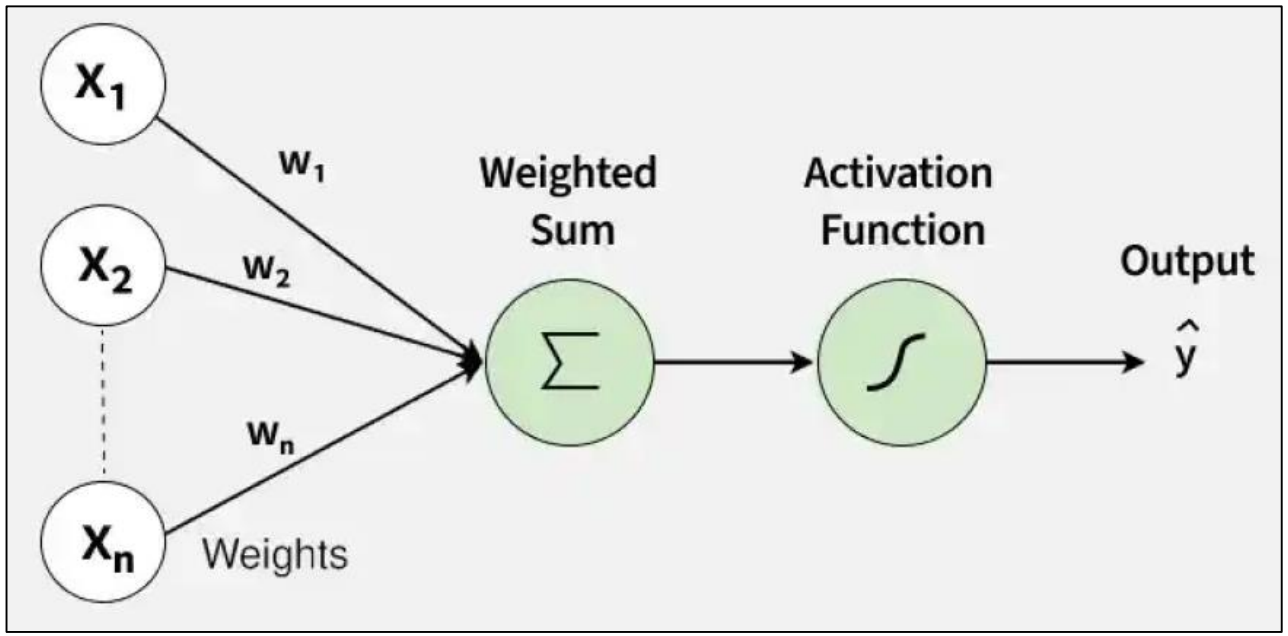
TensorFlow vs Other Frameworks

Feature	TensorFlow	PyTorch	Keras	Scikit-Learn
Primary Focus	Deep learning and production	Research & experimentation	High-level DL API (on TensorFlow)	Traditional ML (SVM, trees, regression)
Deployment	Strong (Lite, JS, Serving)	Limited vs TensorFlow	Uses TensorFlow pipeline	Minimal deployment focus
Ease of Use	Moderate, needs setup	Easy & flexible	Very easy (beginner-friendly)	Simple for small ML tasks
Flexibility	Good for prod, less for research	Highly flexible	Limited vs raw TF	Limited (no deep learning)
Use Cases	NLP, CV, RL, production	Research, prototyping	Quick DL model building	Classification, clustering, regression
Neural Networks	Strong (CNN, RNN, RL)	Strong (RNN, GAN, LSTM)	Simplified NN building	Not for deep learning
Learning Curve	Steep but powerful	Easier (research), complex for prod	Easiest	Easy
Community	Large and growing	Growing, research-focused	Part of TF ecosystem	Large (classical ML only)

Single Layer Perceptron in TensorFlow

A Single Layer Perceptron (SLP) is inspired by biological neurons and their ability to process information. It is based on the concept of an artificial neuron, which acts as the basic building block of neural networks and processes inputs to produce an output.

- Receive input signals from external sources.
- Process the inputs and decide whether to activate or not.
- Send the output signal to the next neuron or system.



Implementation

Step1: Import required libraries

- Scikit-learn is used for data handling and simple machine learning utilities.
- TensorFlow is used to build and train the neural network model.

```
import tensorflow as tf
from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
```

Step 2: Create and split dataset

Create a simple dataset with two features for binary classification and split it into training and testing sets.

```
X, y = make_classification(
    n_samples=1000,
    n_features=2,
    n_informative=2,
    n_redundant=0,
    n_repeated=0,
    n_classes=2,
    random_state=42
)

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

Step 3: Standardize the Dataset

Standardize the dataset so that features are on the same scale, which helps the model train faster and improves performance.

```
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

Step 4: Building a neural network

Build a single layer perceptron using a Sequential model with one Dense layer. The sigmoid activation is used to produce outputs between 0 and 1 for binary classification.

```
model = tf.keras.Sequential([
    tf.keras.layers.Dense(1, activation='sigmoid', input_shape=(2,))
])
```

Step 5: Compile the Model

Compile the model by selecting an optimizer, loss function and evaluation metric suitable for binary classification.

```
model.compile(optimizer='adam',
              loss='binary_crossentropy',
              metrics=['accuracy'])
```

Step 6: Train the Model

Train the model on the training data using epochs and batch size, while keeping a validation split to monitor performance.

```
history = model.fit(X_train, y_train,
                    epochs=50,
                    batch_size=16,
                    validation_split=0.1,
                    verbose=0)
```

Step 7: Model Evaluation

Evaluate the model on test data to check its performance on unseen samples.

```
loss, accuracy = model.evaluate(X_test, y_test, verbose=0)
print(f"Test Accuracy: {accuracy:.2f}")
```

Output: *Test Accuracy: 0.88*

Advantages

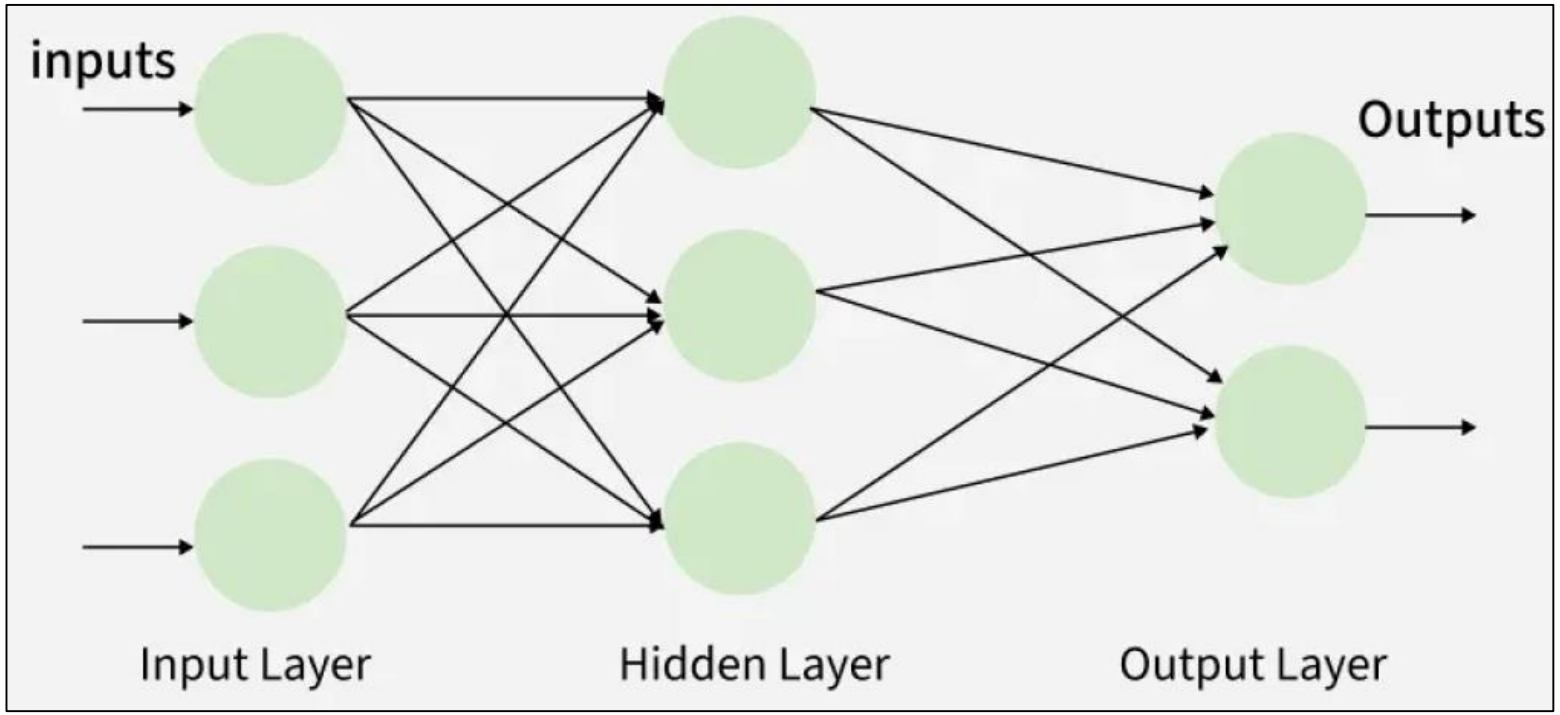
- Simple and easy to implement, making it ideal for beginners.
- Requires less computational power compared to complex neural networks.
- Works well for linearly separable data.
- Fast training due to a single layer and fewer parameters.

Limitations

- Can only solve linearly separable problems.
- Cannot capture complex patterns in data.
- Limited performance compared to multi layer neural networks.
- Less effective for real world problems with high complexity.

Multi-Layer Perceptron Learning in Tensorflow

Multi-Layer Perceptron (MLP) consists of fully connected dense layers that transform input data from one dimension to another. It is called multi-layer because it contains an input layer, one or more hidden layers and an output layer. The purpose of an MLP is to model complex relationships between inputs and outputs.



- **Input Layer**: Each neuron or node in this layer corresponds to an input feature. For instance, if you have three input features the input layer will have three neurons.
- **Hidden Layers**: MLP can have any number of hidden layers with each layer containing any number of nodes. These layers process the information received from the input layer.
- **Output Layer**: The output layer generates the final prediction or result. If there are multiple outputs, the output layer will have a corresponding number of neurons.

Every connection in the diagram is a representation of the fully connected nature of an MLP. This means that every node in one layer connects to every node in the next layer. As the data moves through the network each layer transforms it until the final output is generated in the output layer.

Working of Multi-Layer Perceptron

1. Forward Propagation

In forward propagation the data flows from the input layer to the output layer, passing through any hidden layers. Each neuron in the hidden layers processes the input as follows:

Weighted Sum: The neuron computes the weighted sum of the inputs:

$$z = \sum_i w_i x_i + b$$

Where:

- x_i is the input feature.
- w_i is the corresponding weight.
- b is the bias term.

Activation Function: The weighted sum z is passed through an activation function to introduce non-linearity. Common activation functions include:

- **Sigmoid**: $\sigma(z) = \frac{1}{1+e^{-z}}$
- **ReLU (Rectified Linear Unit)**: $f(z) = \max(0, z)$
- **Tanh (Hyperbolic Tangent)**: $\tanh(z) = \frac{2}{1+e^{-2z}} - 1$

2. Loss Function

Once the network generates an output the next step is to calculate the loss using a loss function. In supervised learning this compares the predicted output to the actual label.

For a classification problem the commonly used binary cross-entropy loss function is:

$$L = -\frac{1}{N}\sum_{i=1}^{N}\left[y_i \log(\hat{y}_i) + (1-y_i)\log(1-\hat{y}_i)\right]$$

Where:

- y_i is the actual label.
- $\hat{y}_i$ is the predicted label.
- N is the number of samples.

For regression problems the mean squared error (MSE) is often used:

$$MSE = \frac{1}{N}\sum_{i=1}^{N}(y_i - \hat{y}_i)^2$$

3. Backpropagation

The goal of training an MLP is to minimize the loss function by adjusting the network's weights and biases. This is achieved through backpropagation:

1. **Gradient Calculation**: The gradients of the loss function with respect to each weight and bias are calculated using the chain rule of calculus.
2. **Error Propagation**: The error is propagated back through the network, layer by layer.
3. **Gradient Descent**: The network updates the weights and biases by moving in the opposite direction of the gradient to reduce the loss: $w = w - \eta \cdot \frac{\partial L}{\partial w}$

Where:

- w is the weight.
- η is the learning rate.
- $\frac{\partial L}{\partial w}$ is the gradient of the loss function with respect to the weight.

4. Optimization

MLPs rely on optimization algorithms to iteratively refine the weights and biases during training. Popular optimization methods include:

- Stochastic Gradient Descent (SGD): Updates the weights based on a single sample or a small batch of data:

$$w = w - \eta \cdot \frac{\partial L}{\partial w}$$

- Adam Optimizer: An extension of SGD that incorporates momentum and adaptive learning rates for more efficient training:
  - $m_t = \beta_1 m_{t-1} + (1 - \beta_1) \cdot g_t$
  - $v_t = \beta_2 v_{t-1} + (1 - \beta_2) \cdot g_t^2$

  Here g_t represents the gradient at time t and β_1, β_2 are decay rates.

Implementing Multi Layer Perceptron

1. Importing Modules and Loading Dataset

First we import necessary libraries such as TensorFlow, NumPy and Matplotlib for visualizing the data. We also load the MNIST dataset.

```
import tensorflow as tf
import numpy as np
import matplotlib.pyplot as plt
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Flatten, Dense

(x_train, y_train), (x_test, y_test) = tf.keras.datasets.mnist.load_data()
```

2. Loading and Normalizing Image Data

Next we normalize the image data by dividing by 255 (since pixel values range from 0 to 255) which helps in faster convergence during training.

2. Loading and Normalizing Image Data

Next we normalize the image data by dividing by 255 (since pixel values range from 0 to 255) which helps in faster convergence during training.

```
gray_scale = 255

x_train = x_train.astype('float32') / gray_scale
x_test = x_test.astype('float32') / gray_scale

print("Feature matrix (x_train):", x_train.shape)
print("Target matrix (y_train):", y_train.shape)
print("Feature matrix (x_test):", x_test.shape)
print("Target matrix (y_test):", y_test.shape)
```

Output:

```
Feature matrix (x_train): (60000, 28, 28)
Target matrix (y_train): (60000,)
Feature matrix (x_test): (10000, 28, 28)
Target matrix (y_test): (10000,)
```

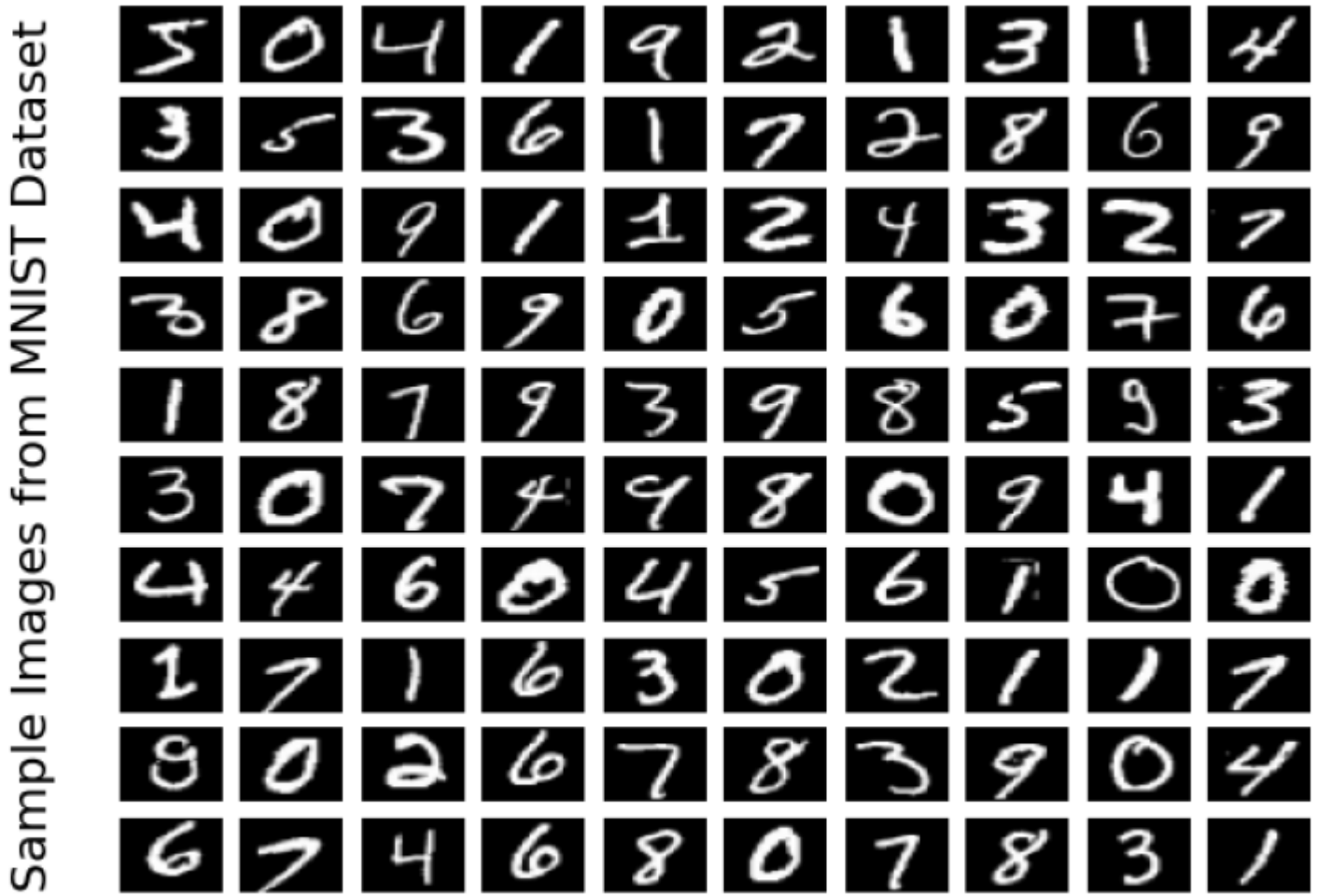
Multi-Layer Perceptron Learning in Tensorflow

3. Visualizing Data

To understand the data better we plot the first 100 training samples each representing a digit.

```
fig, ax = plt.subplots(10, 10)
k = 0
for i in range(10):
    for j in range(10):
        ax[i][j].imshow(x_train[k].reshape(28, 28), aspect='auto')
        k += 1
plt.show()
```

Output:



Multi-Layer Perceptron Learning in Tensorflow

4. Building the Neural Network Model

Here we build a Sequential neural network model. The model consists of:

- **Flatten Layer**: Reshapes 2D input (28x28 pixels) into a 1D array of 784 elements.
- **Dense Layers**: Fully connected layers with 256 and 128 neurons, both using the relu-activation function.
- **Output Layer**: The final layer with 10 neurons representing the 10 classes of digits (0-9) with sigmoid activation.

```
model = Sequential([
    Flatten(input_shape=(28, 28)),
    Dense(256, activation='sigmoid'),
    Dense(128, activation='sigmoid'),
    Dense(10, activation='softmax'),
])
```

5. Compiling the Model

Once the model is defined we compile it by specifying:

```
model.compile(optimizer='adam',
              loss='sparse_categorical_crossentropy',
              metrics=['accuracy'])
```

- **Optimizer**: Adam for efficient weight updates.
- **Loss Function**: Sparse categorical cross entropy, which is suitable for multi-class classification.
- **Metrics**: Accuracy to evaluate model performance.

6. Training the Model

We train the model on the training data using 10 epochs and a batch size of 2000. We also use 20% of the training data for validation to monitor the model's performance on unseen data during training.

```
mod = model.fit(x_train, y_train, epochs=10,
          batch_size=2000,
          validation_split=0.2)

print(mod)
```

Output:

```
Epoch 1/10
24/24 ━━━━━━━━━━━━━━━━━━━━ 2s 73ms/step - accuracy: 0.9232 - loss: 0.2767 - val_accuracy: 0.9276 - val_loss: 0.2544
Epoch 2/10
24/24 ━━━━━━━━━━━━━━━━━━━━ 2s 60ms/step - accuracy: 0.9282 - loss: 0.2558 - val_accuracy: 0.9312 - val_loss: 0.2439
Epoch 3/10
24/24 ━━━━━━━━━━━━━━━━━━━━ 1s 55ms/step - accuracy: 0.9317 - loss: 0.2422 - val_accuracy: 0.9334 - val_loss: 0.2333
Epoch 4/10
24/24 ━━━━━━━━━━━━━━━━━━━━ 2s 77ms/step - accuracy: 0.9332 - loss: 0.2334 - val_accuracy: 0.9362 - val_loss: 0.2244
Epoch 5/10
24/24 ━━━━━━━━━━━━━━━━━━━━ 2s 87ms/step - accuracy: 0.9373 - loss: 0.2242 - val_accuracy: 0.9385 - val_loss: 0.2164
Epoch 6/10
24/24 ━━━━━━━━━━━━━━━━━━━━ 2s 59ms/step - accuracy: 0.9371 - loss: 0.2170 - val_accuracy: 0.9400 - val_loss: 0.2098
Epoch 7/10
24/24 ━━━━━━━━━━━━━━━━━━━━ 1s 53ms/step - accuracy: 0.9387 - loss: 0.2093 - val_accuracy: 0.9430 - val_loss: 0.2011
Epoch 8/10
24/24 ━━━━━━━━━━━━━━━━━━━━ 3s 54ms/step - accuracy: 0.9433 - loss: 0.1974 - val_accuracy: 0.9438 - val_loss: 0.1961
Epoch 9/10
24/24 ━━━━━━━━━━━━━━━━━━━━ 1s 53ms/step - accuracy: 0.9456 - loss: 0.1909 - val_accuracy: 0.9455 - val_loss: 0.1906
Epoch 10/10
24/24 ━━━━━━━━━━━━━━━━━━━━ 1s 60ms/step - accuracy: 0.9466 - loss: 0.1858 - val_accuracy: 0.9488 - val_loss: 0.1840
```

Multi-Layer Perceptron Learning in Tensorflow

7. Evaluating the Model

After training we evaluate the model on the test dataset to determine its performance.

```
results = model.evaluate(x_test, y_test, verbose=0)
print('Test loss, Test accuracy:', results)
```

Output:

Test loss, Test accuracy: [0.2682029604911804, 0.9257000088691711]

We got the accuracy of our model 92% by using model.evaluate() on the test samples.

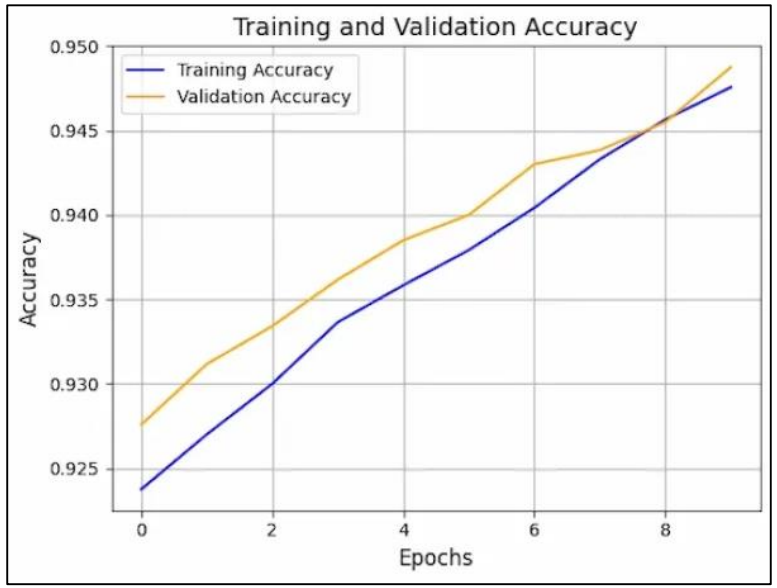
8. Visualizing Training and Validation Loss VS Accuracy

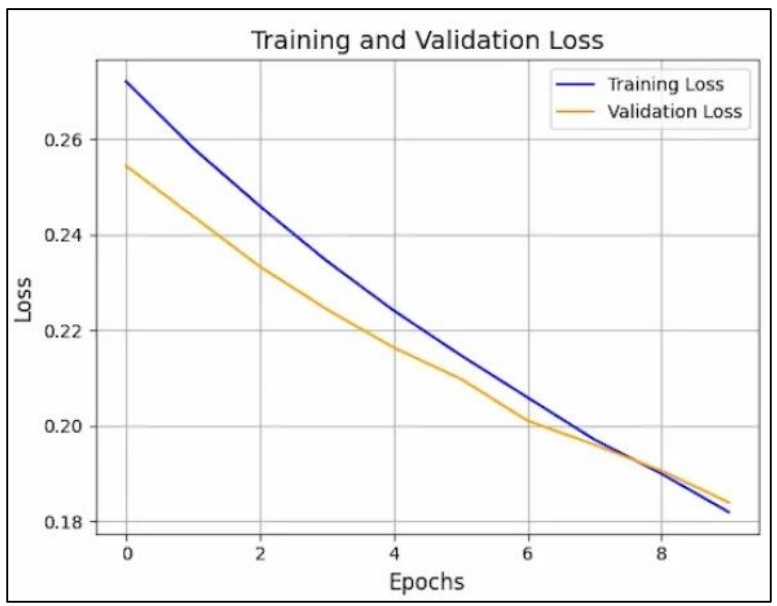
```
plt.figure(figsize=(12, 5))

plt.subplot(1, 2, 1)
plt.plot(mod.history['accuracy'], label='Training Accuracy', color='blue')
plt.plot(mod.history['val_accuracy'],
         label='Validation Accuracy', color='orange')
plt.title('Training and Validation Accuracy', fontsize=14)
plt.xlabel('Epochs', fontsize=12)
plt.ylabel('Accuracy', fontsize=12)
plt.legend()
plt.grid(True)

plt.subplot(1, 2, 2)
plt.plot(mod.history['loss'], label='Training Loss', color='blue')
plt.plot(mod.history['val_loss'], label='Validation Loss', color='orange')
plt.title('Training and Validation Loss', fontsize=14)
plt.xlabel('Epochs', fontsize=12)
plt.ylabel('Loss', fontsize=12)
plt.legend()
plt.grid(True)

plt.suptitle("Model Training Performance", fontsize=16)
plt.tight_layout()
plt.show()
```





Model Training Performance

Advantages

- **Versatility**: MLPs can be applied to a variety of problems, both classification and regression.
- **Non-linearity**: Using activation functions MLPs can model complex, non-linear relationships in data.
- **Parallel Computation**: With the help of GPUs, MLPs can be trained quickly by takfing advantage of parallel computing.

Disadvantages

- **Computationally Expensive**: MLPs can be slow to train especially on large datasets with many layers.
- **Prone to Overfitting**: Without proper regularization techniques they can overfit the training data leading to poor generalization.
- **Sensitivity to Data Scaling**: They require properly normalized or scaled data for optimal performance.